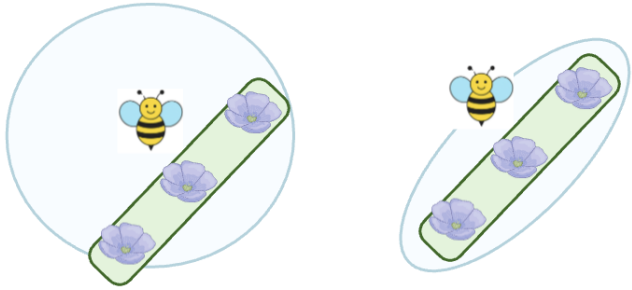
CHARITÉ
UNIVERSITÄTSMEDIZIN BERLIN



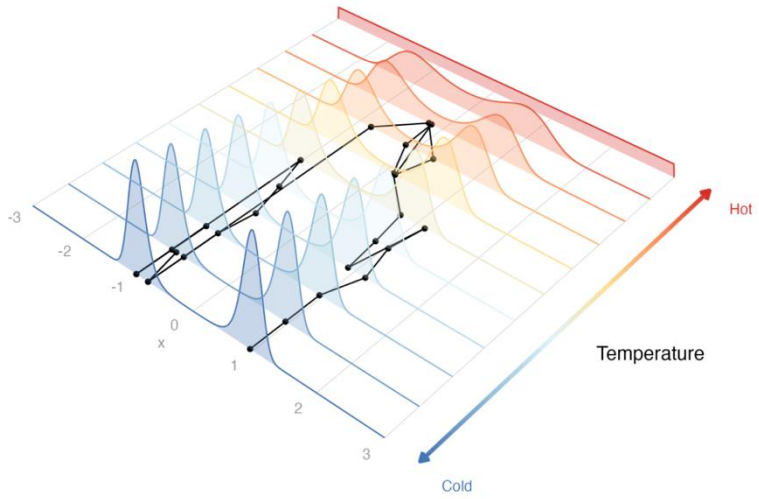
A whirlwind tour of MCMC samplers and diagnostics

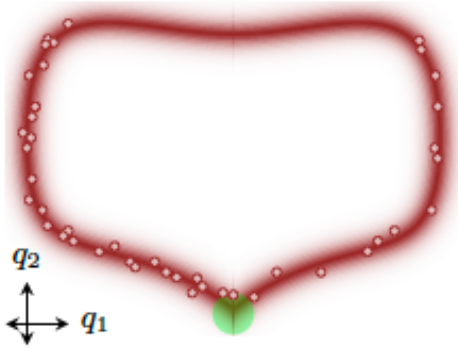
David Hodgson, Charité — Universitätsmedizin Berlin

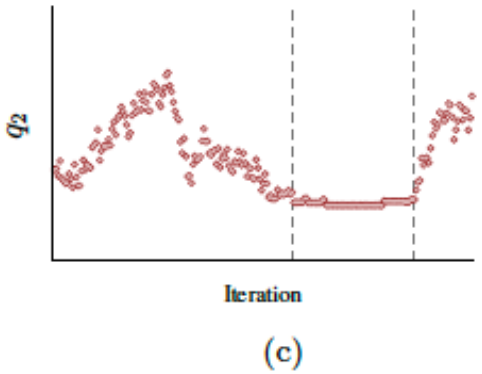
BIDDE

14/07/26

https://dchodge.github.io/mcmc-widget-teaching/







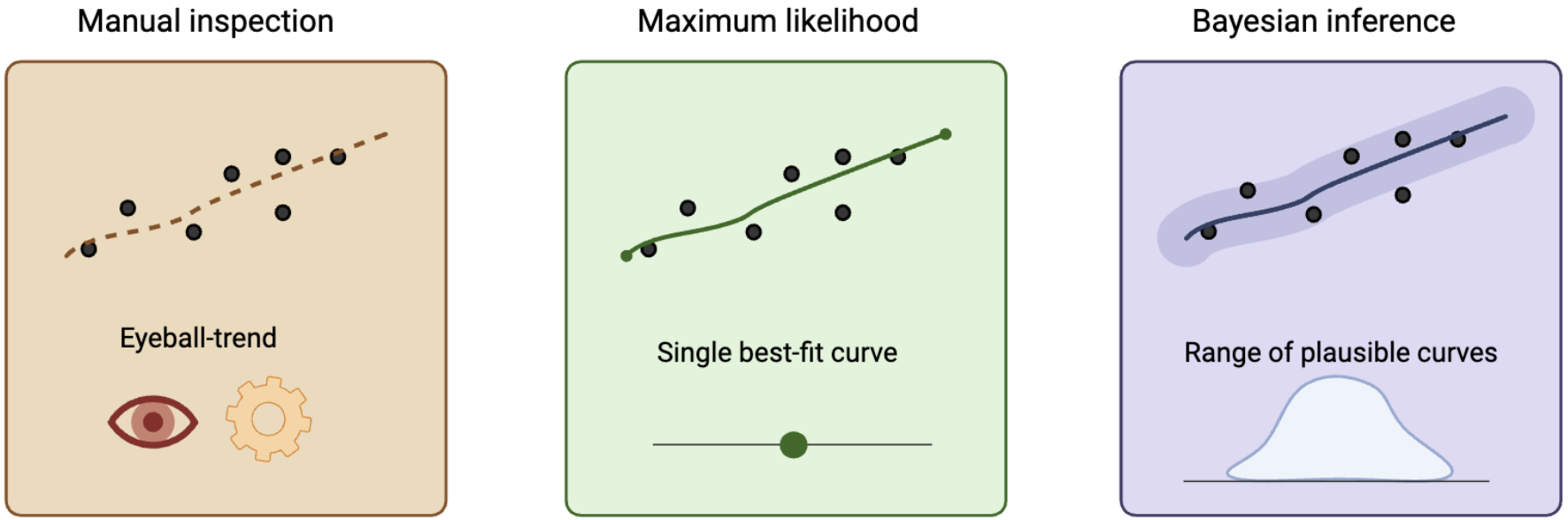
CHARITÉ
UNIVERSITÄTSMEDIZIN BERLIN

1. What are MCMC samplers why do we need them?

Follow this link to follow through the examples

We often build a model with free parameters, and then once we get data, we want to know what parameters values best fit that data

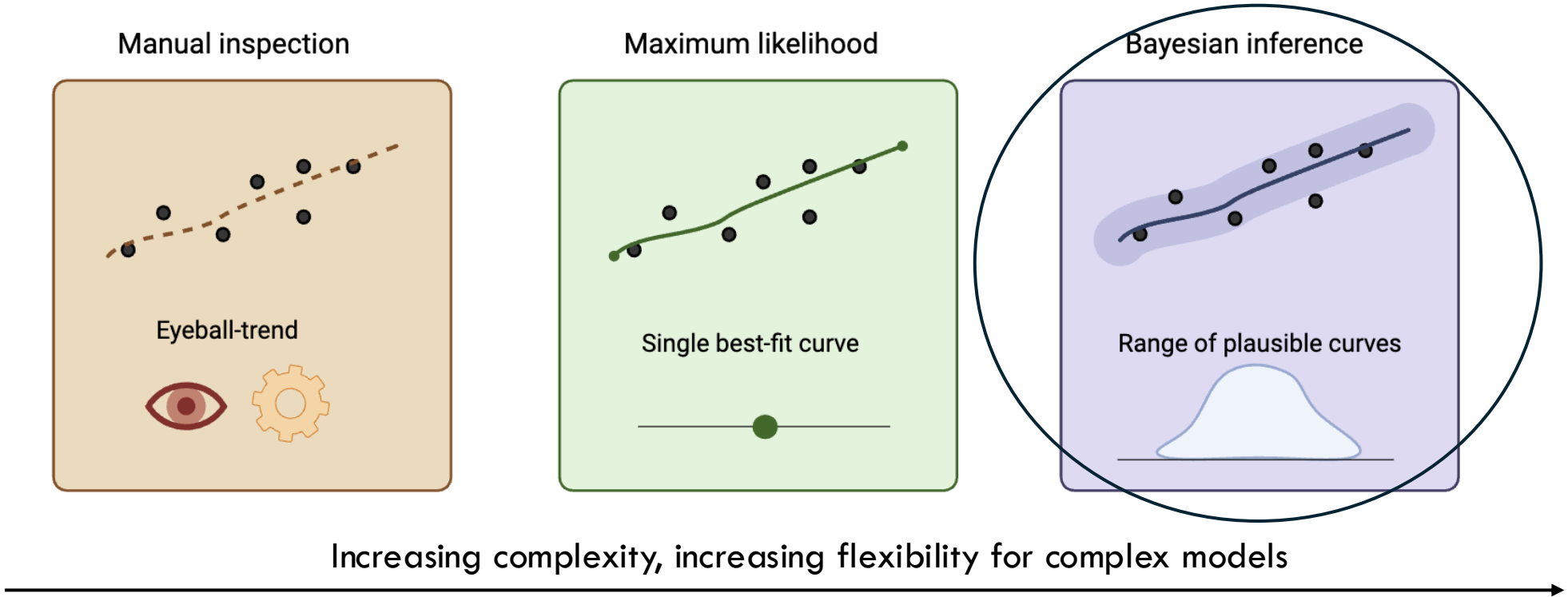
To do this we need to define some metric for what is a "good fit".
This can range from being very primitive to very complicated:



Increasing complexity, increasing flexibility for complex models

We often build a model with free parameters, and then once we get data, we want to know what parameters values best fit that data

To do this we need to define some metric for what is a "good fit".
This can range from being very primitive to very complicated:



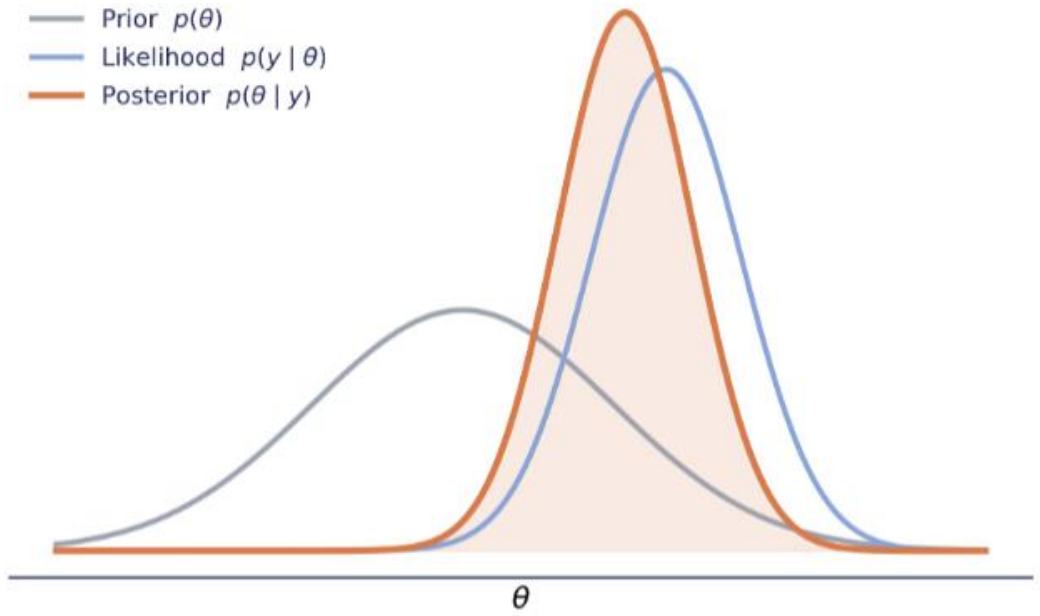
Bayes' rule, set up

posterior $\propto$ *likelihood* $\times$ *prior*

y, data
θ, fitted parameters

$$\pi(\theta|y) = \frac{p(y|\theta)p(\theta)}{p(y)}$$

$$p(y) = \int p(y|\theta)p(\theta)\, d\theta$$



Prior and likelihood combine to form the posterior

Posterior $\pi(\theta|y)$

Likelihood $p(y|\theta)$

Prior $p(\theta)$

Challenge: turning the $\propto$ to $=$ means diving by the normalising constant $p(y)$ which means integrating over θ. This is not tractable usually, so we have to compute it: approximating it is the rest of this talk.

Conjugate prior: Estimating the probability of a female birth

Historically; probability of a female birth in Europe $p < 0.5$.
Assuming each birth is independent. Let θ be the proportion of births that are female, y be the number of girls born in n recorded births.
Define the likelihood as binomial, uniform prior on the probability of being female:

$$p(y|\theta) = Bin(y|n, \theta = p) = \binom{n}{y} p^y (1-p)^{n-y}$$

$$p(\theta) \sim U(0,1) = 1$$

$$\pi(\theta|y) = \frac{p(y|\theta)p(\theta)}{p(y)} = \frac{\binom{n}{y} p^y (1-p)^{n-y} \cdot 1}{\int_0^1 \binom{n}{y} p^y (1-p)^{n-y} d\theta}$$

$$= Beta(y+1, n-y+1)$$

Analytical solution!

Sometimes you can have a direct functional form for the posterior; thus make new predictions based on it.

NOTE: The true posterior (or invariant distribution) $\pi(\theta \mid \text{data})$ is a fixed mathematical object.
It exists the instant you specify a prior and a likelihood.
In the Beta-Binomial case, it has a closed form and a general function.

1. **Worked solution:** Gelman, Carlin, Stern, Dunson, Vehtari & Rubin, *Bayesian Data Analysis* (3rd ed.), Chapter 2
2. **History of problem and Laplaces approach:** https://georgemaciunas.com/exhibitions/knowledge-as-art-chance-computability-and-improving-education-thomas-bayes-alan-turing-george-maciunas/thomas-bayes/a-history-of-bayes-theorem/

Some theory on Monte Carlo samples

WATCH:
https://www.youtube.com/watch?v=lY4rSeX8IL4

Sampled values $\theta_1, \ldots, \theta_N$ are a *random, finite* and one realisation of a stochastic process. The **distribution:**
$\hat{\pi}_N(\theta) = \frac{1}{N}\sum_{i=1}^{N} \delta\,(\theta - \theta_i)$

This is an *estimator* of $\pi(\theta \mid \text{data})$; but not necessarily equal. For them to be equal two concepts:

1. Strong Law of Large Numbers $\hat{\pi}_N \xrightarrow{\text{a.s.}} \mathbb{E}[\pi(\theta)]$ as $N \to \infty$
Requires;

- θ_i indentically independent variable and also finite.

2. Central Limit Theorem (rate of convergence)
If additionally $\text{Var}\big(\pi(\theta)\big)$ *is finite*

$$\sqrt{N}(\hat{\pi}_N - \mathbb{E}[\pi(\theta)]) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$$

This is why your 90% interval shrinks like $1/\sqrt{N}$— quadrupling draws halves the interval width.

Basically, if you randomly select inputs and everything is finite, the samples converge to the distribution as you pick more samples (feels obvious but important result)

Mechanistic model: for the probability of a female birth

Mechanistic drivers of female birth rate

- *E*, Y-sperm fertilization edge (motility/timing bias toward Y)
- *L*, Excess male embryo loss rate (XY more likely)
- *S*, Population stress / shock level (male fetus more fragile and prone to miscarriage)
- *A*, Age (older age = higher chance of daughters)

$$\theta = (E, L, S, A)$$

E, L, S, A have priors, p(E), p(L), p(S), p(A)

$$p = F_c(\theta)$$

Could be a solution of an ODE, ABM, etc. Nonlinear

$$p(y|\theta) = Bin(y|n, \theta) = \binom{n}{y} F_c(\theta)^y (1 - F_c(\theta))^{n-y}$$

No closed form for posterior like before

$$p(y) = \int p(y|\theta)p(\theta)\, d\delta$$

Sampling model: for the probability of a female birth

$$p(y|\theta) = Bin(y|n,\theta) = \binom{n}{y} F_c(\theta)^y (1 - F_c(\theta))^{n-y}$$

Problem 1: no closed form for $F_c(\theta)$ is a numerical ODE solution. Thus p(δ | y) has no algebraic expression

Problem 2: intractable denominator p(y) = ∫∫∫∫ p(y|δ) p(F)p(L)p(S)p(A) dδ. Even at 100 grid pts per param:100^4 = 100,000,000 ODE solves→ completely infeasible

Sampling model: for the probability of a female birth

$$p(y|\theta) = Bin(y|n,\theta) = \binom{n}{y} F_c(\theta)^y (1 - F_c(\theta))^{n-y}$$

Problem 1: no closed form for $F_c(\delta)$ is a numerical ODE solution. Thus p(δ | y) has no algebraic expression.

Problem 2: intractable denominator p(y) = ∫∫∫∫ p(y|δ) p(F)p(L)p(S)p(A) dδ. Even at 100 grid pts per param:100^4 = 100,000,000 ODE solves→ completely unfeasible

BUT: we can sample from $p(y|\theta)p(\theta)$ if we choose values of θ
Does this help up sample from the true posterior?

Yes but need more rules

Theoretical requirements of the samples; MCMC

Previously we needed Strong Law of Large Numbers and Central Limit Theorem.

For MCMC (sampling a posterior with no closed form), we need to impose stronger requirements to make up for the fact that we cannot calculate the denominator.

Main is the samples must be generate in a special way (chain, X_t) to force **ergodicity.**
Which requires addition that the chain of sampled values is:

Irreducibility — the chain *can* reach everywhere it needs to.
Aperiodicity — the chain doesn't cycle in a way that prevents settling to a single limiting distribution.
Positive recurrence — Improper posterior (prior × likelihood doesn't integrate to something finite)
Detailed balance — $\pi(\theta)P(\theta \to \theta') = \pi(\theta')P(\theta' \to \theta)$, *if* the chain converges to a stationary distribution, that distribution is π. It says nothing about whether the chain actually explores the full space or gets stuck

Together: $X_t \xrightarrow{d} \pi(\mathrm{X} \mid \mathrm{data})$ and the running average converges a.s. to the true posterior expectation.

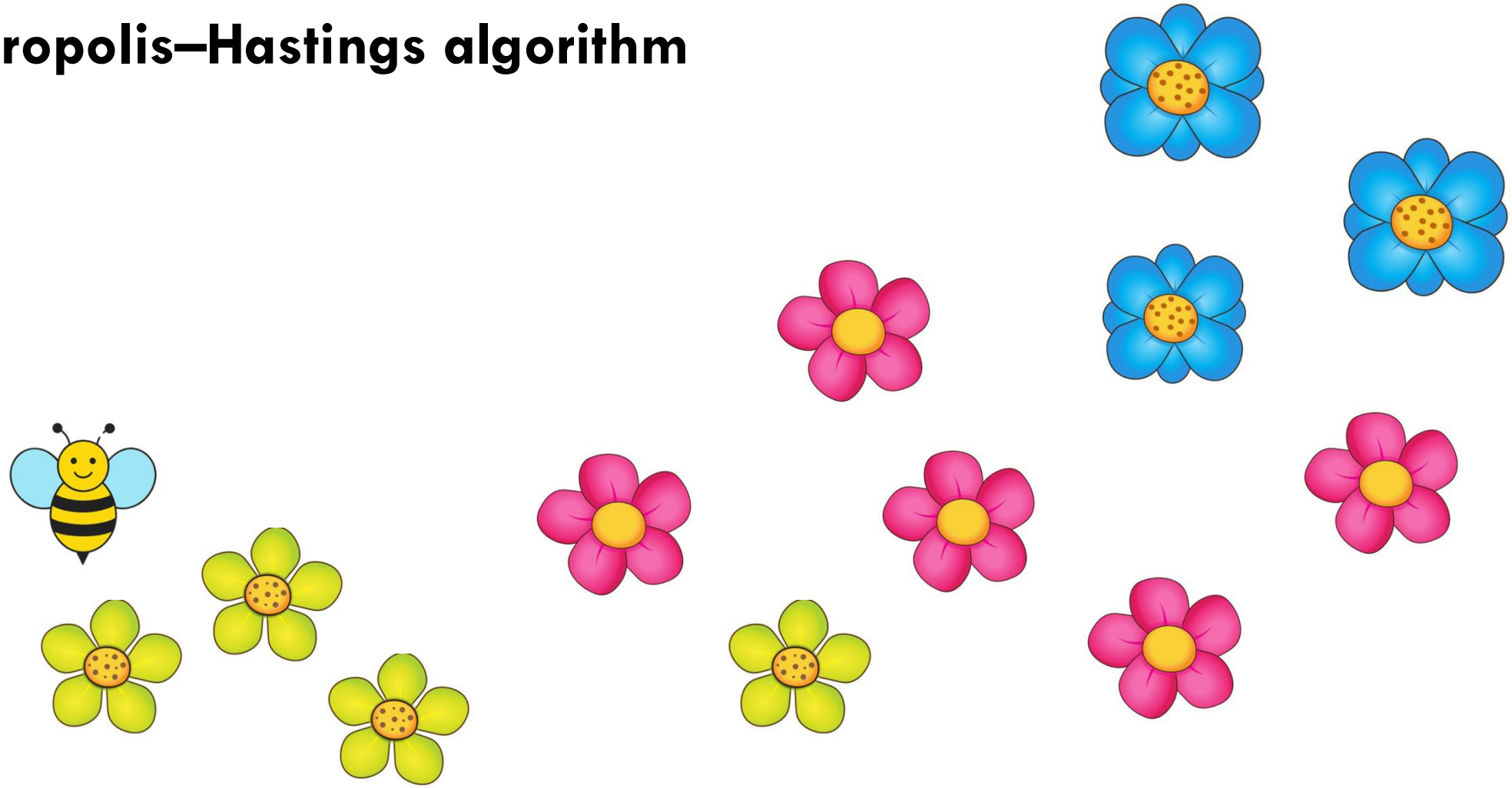
Possible to just sample from $\boldsymbol{p(y|\theta)p(\theta)}$ and still get full posterior distribtuion!!!!
How do we ensure all these complex maths rules are met??

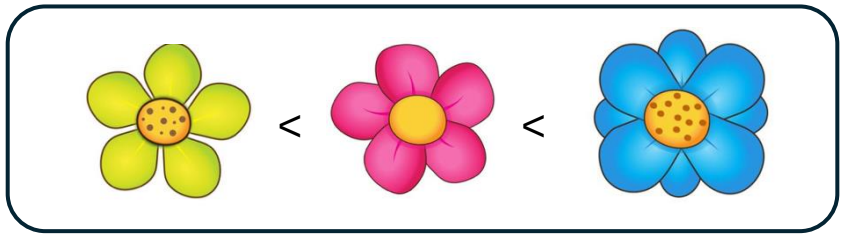
CHARITÉ
UNIVERSITÄTSMEDIZIN BERLIN

2. Meet the family (of samplers)

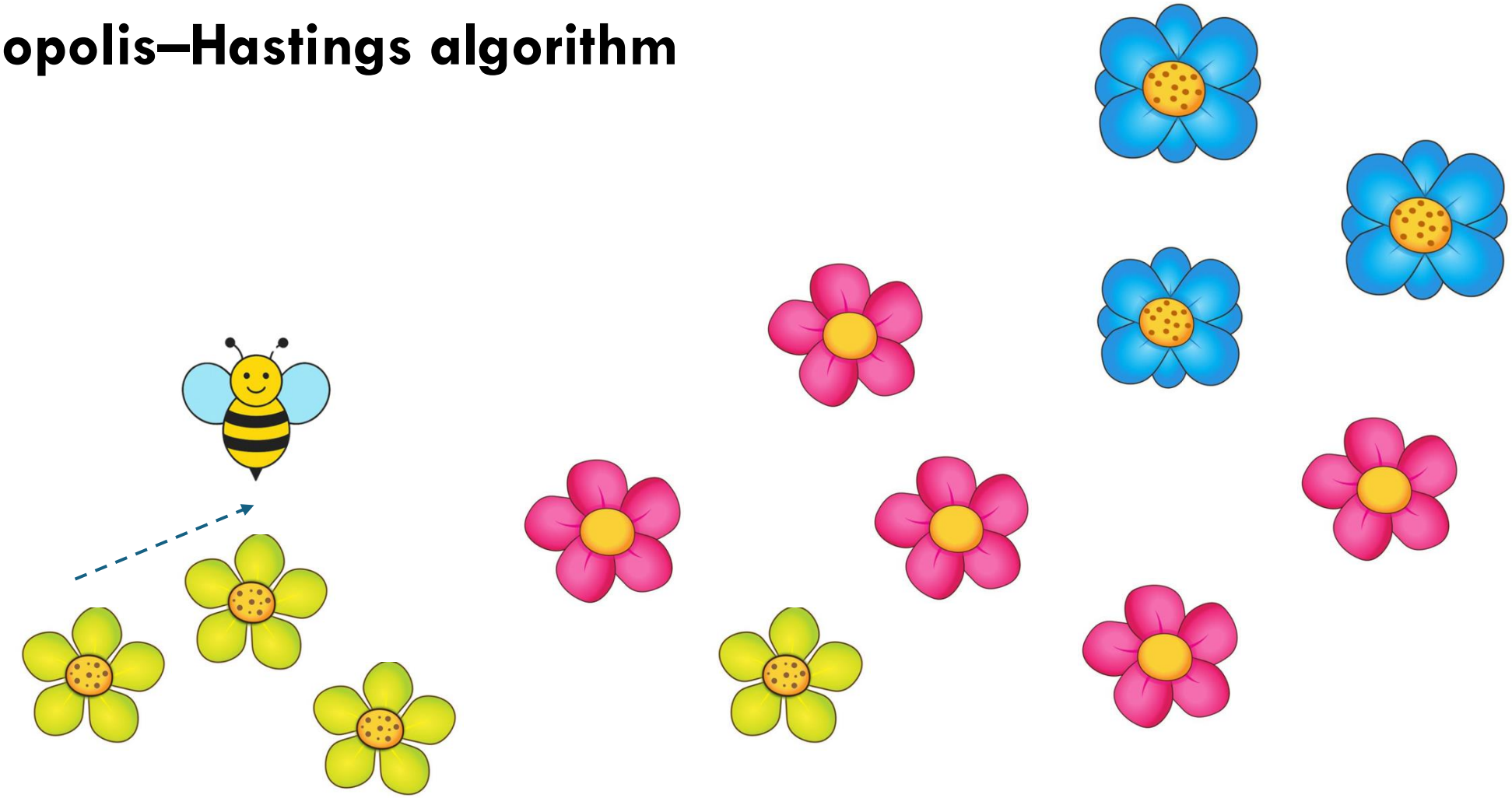
Lots of ways to explore the parameter space effectively

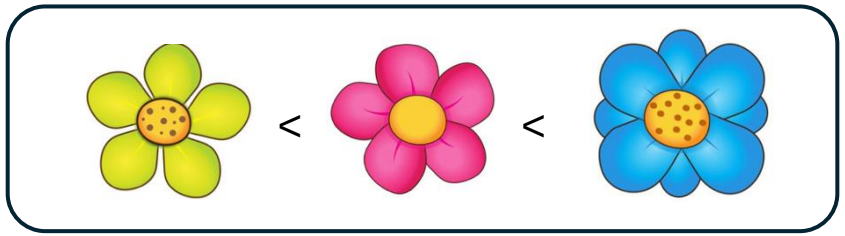
Metropolis–Hastings algorithm



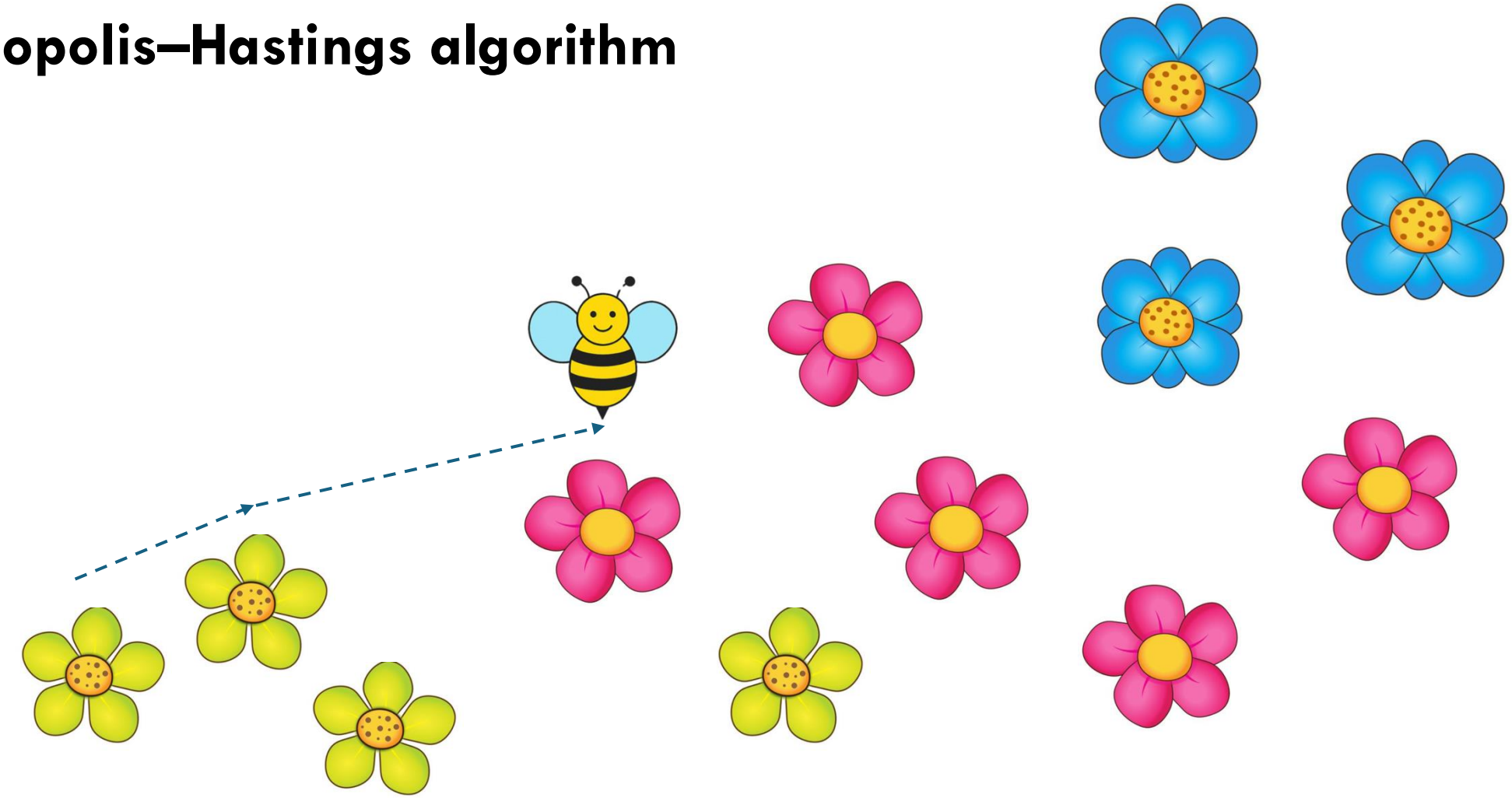


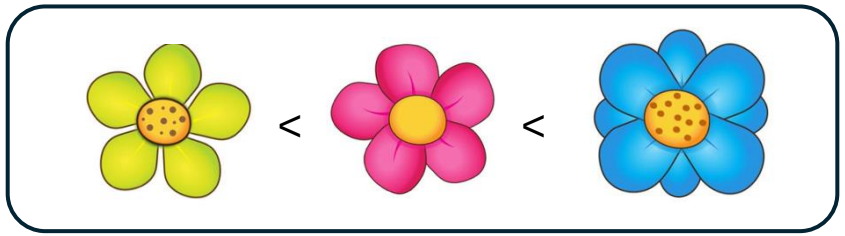
Metropolis–Hastings algorithm



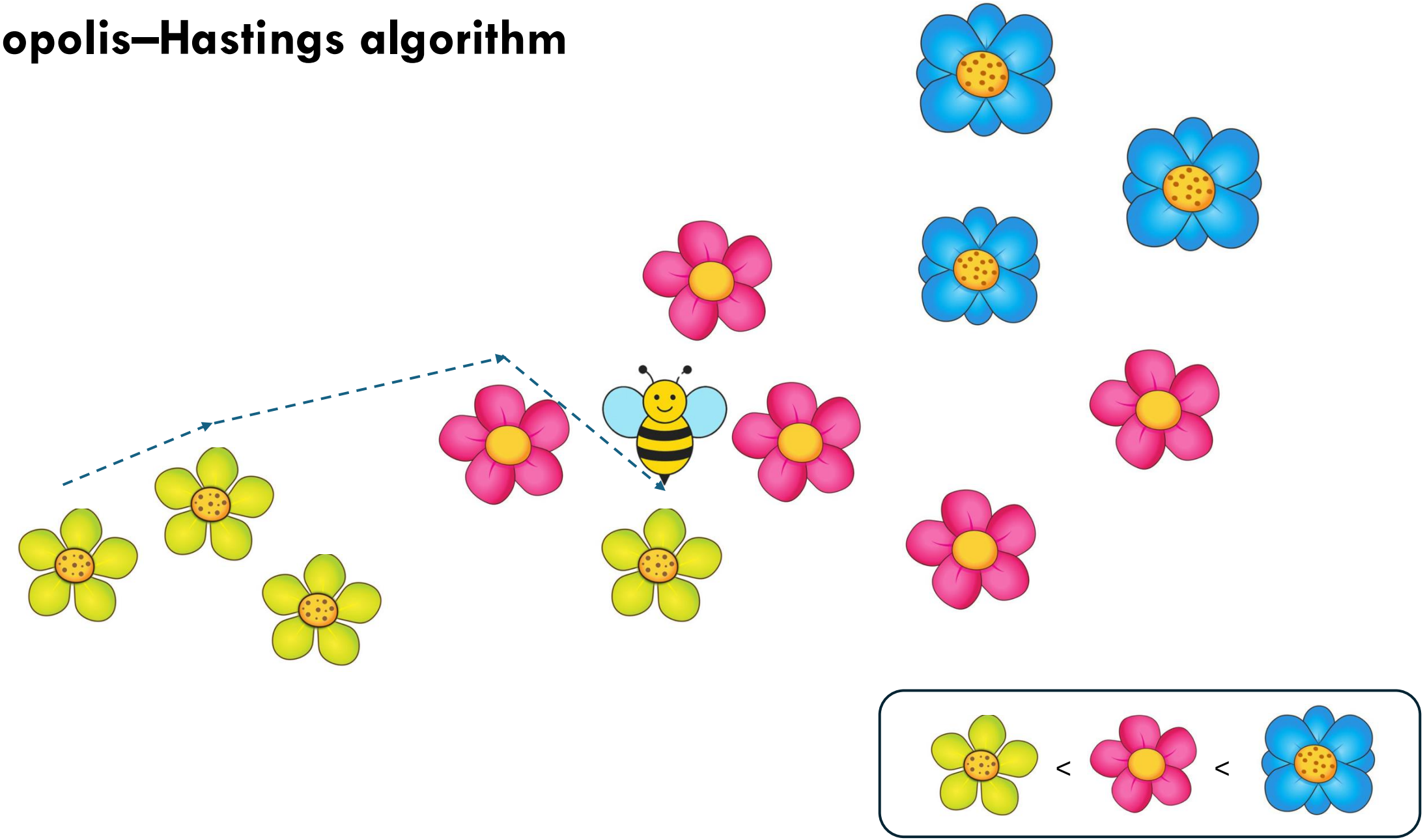


Metropolis–Hastings algorithm

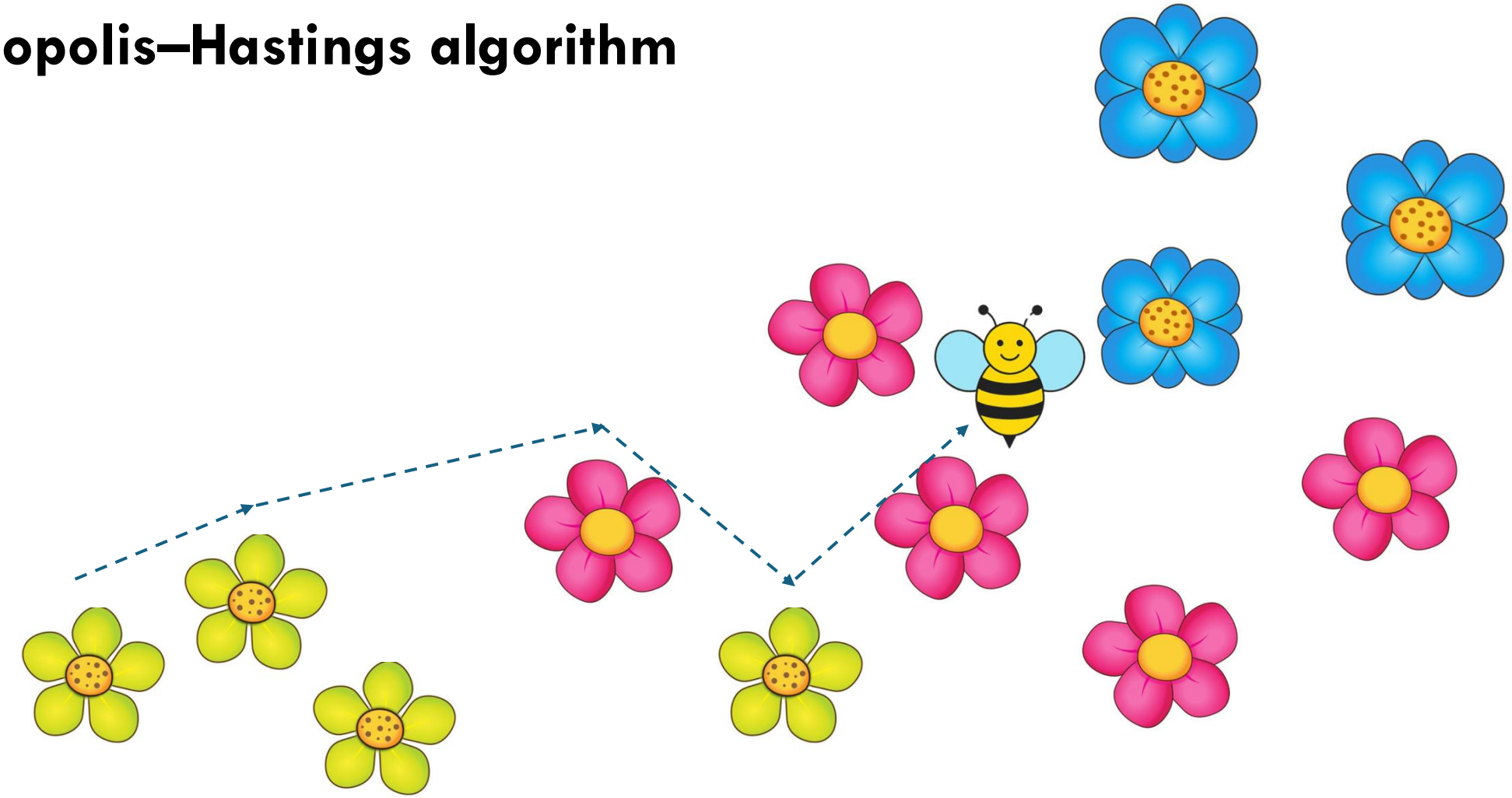


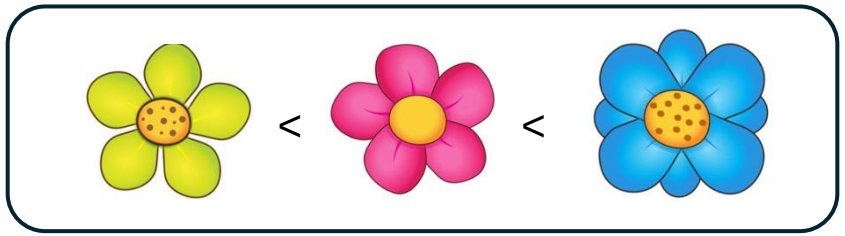


Metropolis–Hastings algorithm

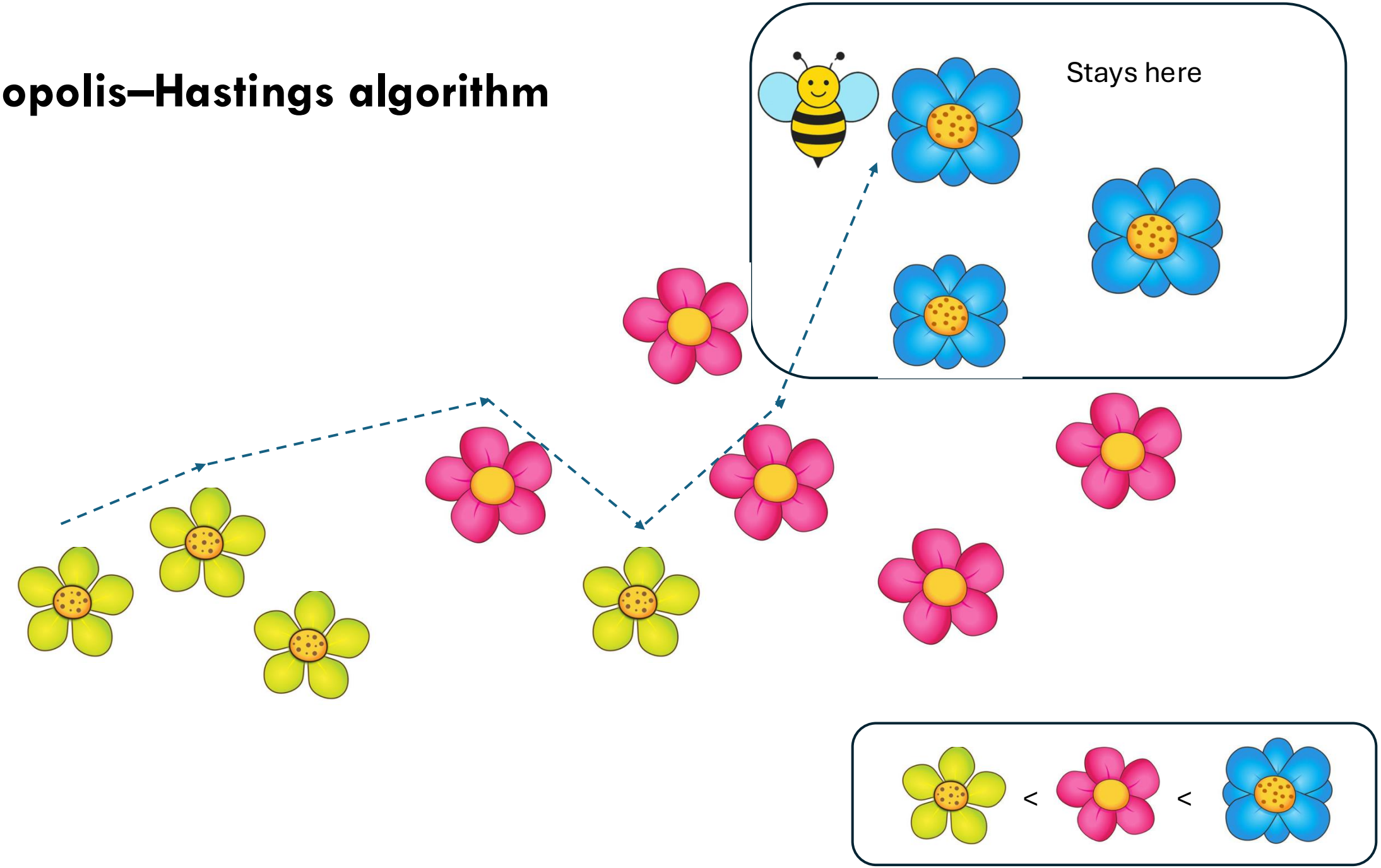


Metropolis–Hastings algorithm





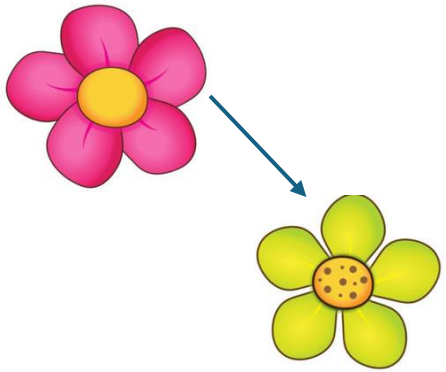
Metropolis–Hastings algorithm



Metropolis–Hastings algorithm (sniff test)

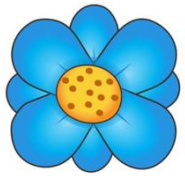
VIDEO:
https://www.youtube.com/watch?v=7IpvsfRgTTY

?



Prob of moving: p( )/ p( ) <1

?



Prob of moving: p( )/ p( ) =1

Only looking locally, thus the global variable p(y) with the complex integral isn't needed

Metropolis–Hastings algorithm (sniff test)

*Poorly chosen q can make **aperiodicity** and **irreducibility** fail*

The algorithm, step by step:

1. **Start somewhere** — pick an initial parameter vector X_0, i = 0
2. **Propose** a move: draw X' from a proposal distribution $q(X'|X_i)$
3. **Compute the acceptance ratio:**

*Not random: this ensures **detailed balance***
No normalising constant needed

$$\alpha = \min\left(1, \frac{p(X' \mid y)}{p(X_i \mid y)}\right) = \min\left(1, \frac{p(y \mid X')p(X')}{p(y \mid X_i)p(X_i)}\right)$$

4. **Accept or reject:** draw u ~ Uniform(0,1). If u < α, $X_{i+1} \leftarrow X'$. Otherwise $X_{i+1} \leftarrow X_i$
5. **Update i** $\leftarrow i + 1$
6. **Back to Step 2**

The bee translation:

- Current position = where bee is sitting
- Proposal = random hop to a nearby flower
- Acceptance ratio = sniff test: is there more pollen there than here?
- Always move to better flowers, sometimes move to worse ones

1. Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, Edward Teller; Equation of State Calculations by Fast Computing Machines. *J. Chem. Phys.* 1 June 1953; 21 (6): 1087–1092. https://doi.org/10.1063/1.1699114
2. Hastings, W.K. (1970). "Monte Carlo Sampling Methods Using Markov Chains and Their Applications." *Biometrika*, *57*(1), 97–109.

Metropolis–Hastings algorithms, different proposals, $q(X'|X_t)$

1. **Random Walk Metropolis (RWM)**

   $X' \sim N(X_t, \sigma)$, or $X' \sim N(X_t, \Sigma_0)$. User selects σ.

2. Independent MH

$X' \sim q(X_t)$, proposal ignores current position entirely.

Tierney, Luke. "Markov Chains for Exploring Posterior Distributions." *The Annals of Statistics*, vol. 22, no. 4, 1994, pp. 1701–28. *JSTOR*, http://www.jstor.org/stable/2242477. Accessed 6 July 2026.

3. Component-wise / Gibbs-within-MH

In multidimensional spaces, update one parameter at a time, holding others fixed.

Gilks, W.R., Richardson, S., & Spiegelhalter, D. (Eds.). (1995). Markov Chain Monte Carlo in Practice (1st ed.). Chapman and Hall/CRC. https://doi.org/10.1201/b14835

4. MALA (Metropolis-Adjusted Langevin Algorithm)

$X' = X_t + (h/2)\nabla \log p(X_t | y) + \sqrt{h} \cdot \varepsilon$

Proposal drifts uphill toward higher probability before adding noise. Based on a discretised Langevin diffusion

Gareth O. Roberts, Richard L. Tweedie "Exponential convergence of Langevin distributions and their discrete approximations," Bernoulli, Bernoulli 2(4), 341-363, (December 1996)

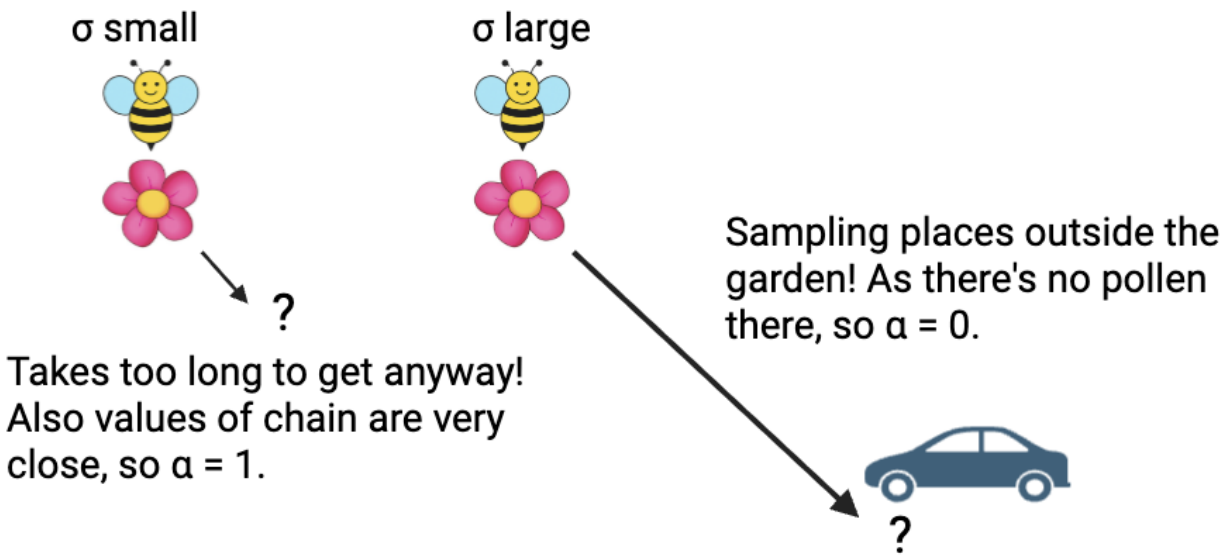
5. Preconditioned Crank-Nicolson (pCN)

$X' = \sqrt{(1-\beta^2)}\, X_t + \beta\, \xi$, where $\xi \sim$ prior

Proposal mixes current position with a fresh prior draw. Acceptance rate doesn't degrade as the dimension grows. Hairer, M., Stuart, A.M. & Vollmer, S.J. (2014). "Spectral Gaps for a Metropolis–Hastings Algorithm in Infinite Dimensions." *Annals of Applied Probability*, 24(6), 2455–2490 — proved pCN's ergodicity properties the following year.

NOTE: In general all of these are not good samplers as they require some a prior definition (step size), one size does not fit all

Adaptive step-size Metropolis–Hastings algorithm



Dynamic update σ for chain number t

If acceptance ratio is > 0.234

σ_t <- σ_{t-1} - γ_t

If acceptance ratio is < 0.234

σ_t <- σ_{t-1} + γ_t

KEY : γ_t must get smaller and tend to 0 as t gets very large
ADAPTIVENESS MUST WEAR OFF

NOTE:

Target acceptance rate of ~23.4% in high dimensions.

Bee analogy: the bee learns how far to hop, but still hops in random directions. Better than fixed step size but still blind to the shape of the pollen patch.

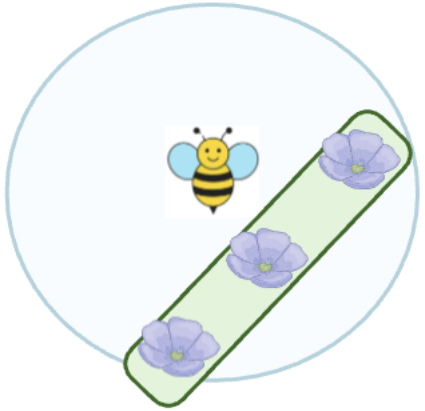
1. Roberts, G.O., Gelman, A. & Gilks, W.R. (1997). "Weak Convergence and Optimal Scaling of Random Walk Metropolis Algorithms." *Annals of Applied Probability*, 7(1), 110–120 — the classic paper establishing the 0.234 optimal acceptance rate result.

Adaptive covariance Metropolis–Hastings algorithm

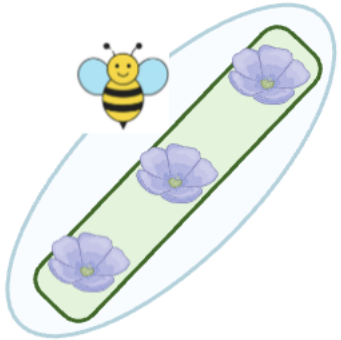
Instead of distance to jump we focus on the shape of the proposal

$$N(\delta_t, \Sigma_0)$$

Uninformed, inefficient



Informed, efficient!



- Covariance of the proposal, Σ_0, can be fixed aprior (hard to know)
- You can also update dynamically using the chain δ_t via the update rule

$$\mu_{i+1} = \mu_i + \gamma_i(X_i - \mu_i)$$

$$\Sigma_{i+1} = \Sigma_i + \gamma_i(X_i - \mu_i)(X_i - \mu_i)^T - \Sigma_i$$

Note: find the average and covariance at each step

The important caveat for adaptive MH:

Adapting the proposal using past samples technically breaks the Markov property — the chain's next step now depends on its whole history, not just the current state. You need to be careful to ensure ergodicity is preserved. The standard fix is to either:

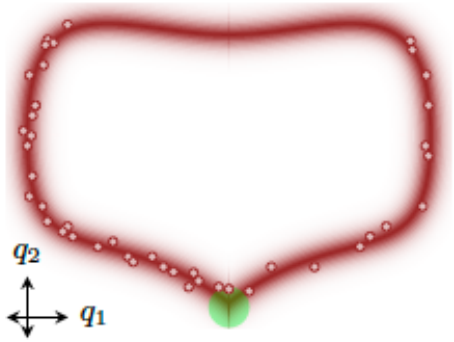
- Freeze adaptation after a burn-in period. So stop dynamically updating after a while.
- Use a diminishing adaptation schedule, this is what γ_i is

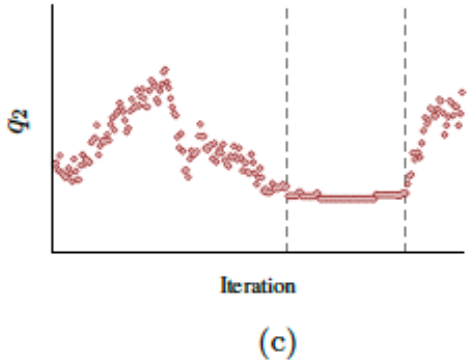
Andrieu, C., Thoms, J. A tutorial on adaptive MCMC. *Stat Comput* **18**, 343–373 (2008). https://doi.org/10.1007/s11222-008-9110-y

Hamiltonian Monte Carlo

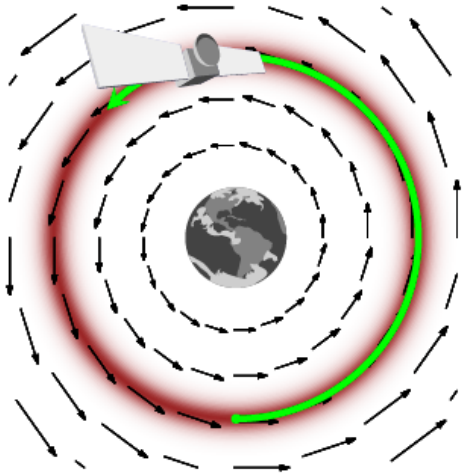
The core problem with random walk and adaptive proposals:

- You're still essentially diffusing through the space which is very slow in high dimensions or spaces with awkward shape





In an example like this it's v hard for our proposals before to explore efficiently



Direction and how fast

The HMC idea:

Borrowed from physics.

In stead of just sampling position we also now sample **momentum.**

Knowing the **momentum** lets the particle build up speed and coast along in a direction. It's *intelligently* moving along the shape of the lanscaope

We are sampling **trajectories** instead of **points**

Betancourt, M. (2017). "A Conceptual Introduction to Hamiltonian Monte Carlo." arXiv:1701.02434

Adaptive Hamiltonian Monte Carlo

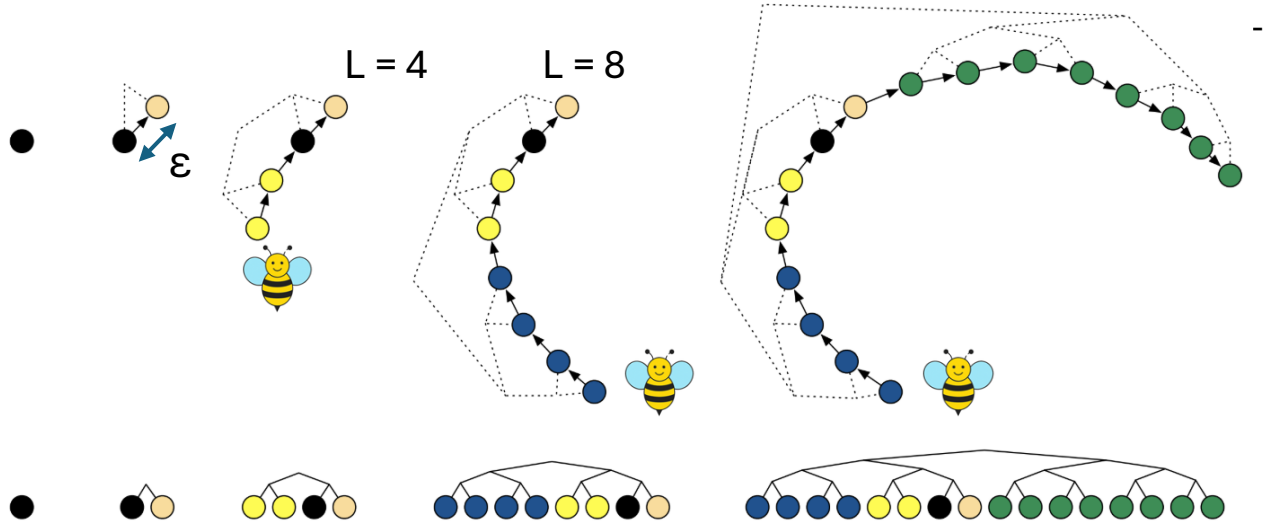
The two things you need to tune:

- ε — leapfrog step size
- L — number of leapfrog steps

Too large ε → discretisation error → low acceptance.

Too small → slow. Too few L steps → still a random walk.

Too many → U-turns (wastes computation).



NUTS — No U-Turn Sampler

Automatically sets L by detecting when the trajectory starts doubling back on itself. Hoffman & Gelman (2014). This is what stan uses under the hood.

Once we have a trajecotory we choose a random point on it and that is one part of the Markov chain. Then repeat.

Hoffman, M.D. & Gelman, A. (2014). "The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo." *Journal of Machine Learning Research*, 15(1), 1593–1623.

Hamiltonian Monte Carlo

HMC SAMPLERS ARE VERY EFFICIENT, SCALES VERY WELL INTO HIGH DIMENSIONS AND HAVE LOTS OF TOOLS TO DIAGNOSE MIXING AND CONVERGENCE!!! BECOMING GOLD STANDARD

!!!CAUTION!!!

For HMC to work the likelihood space needs to be **differentiable everywhere (as momentum uses gradient)**

- Crudely this means the whole surface must be smooth with no degeneracies
- Won't work if
  - You are sampling over discrete spaces (discrete points are not differentiable) latent-variable modelling is often very challenging (possible to integrate out discrete variables)
  - You are fitting a model where there is a rule-based process like a ABM
- Non-linear ODE systems can struggle sometimes, due to the numerical ODE solvers not working well in certain situations. Some observations:

ODE system	HMC feasible?
Small, smooth, non-stiff (2-3 equations)	Yes, works well
Medium nonlinear (5-20 equations)	Marginal, adjoint helps
Large stiff system (50+ equations)	Very difficult

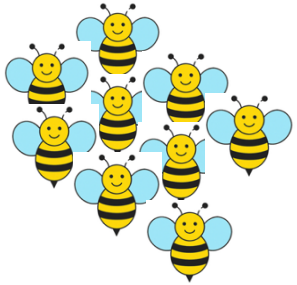
NICE TUTORIAL: https://bayesically-speaking.com/posts/2025-01-21%20hmc-nuts-from-zero/

Population samplers

Core idea behind population samplers is that instead of running one chain, we run a family of chains and let them communicate with each other

Instead of one bee looking for pollen

Imagine a group of bees looking for pollen who chat with each other



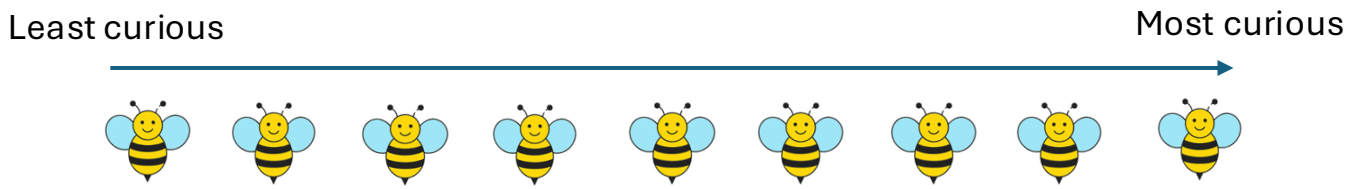
In a population sampler, if one bee gets stuck the other bees can help them leave and explore somewhere else.

There are several population-based samplers, we will focus on **parallel tempering**

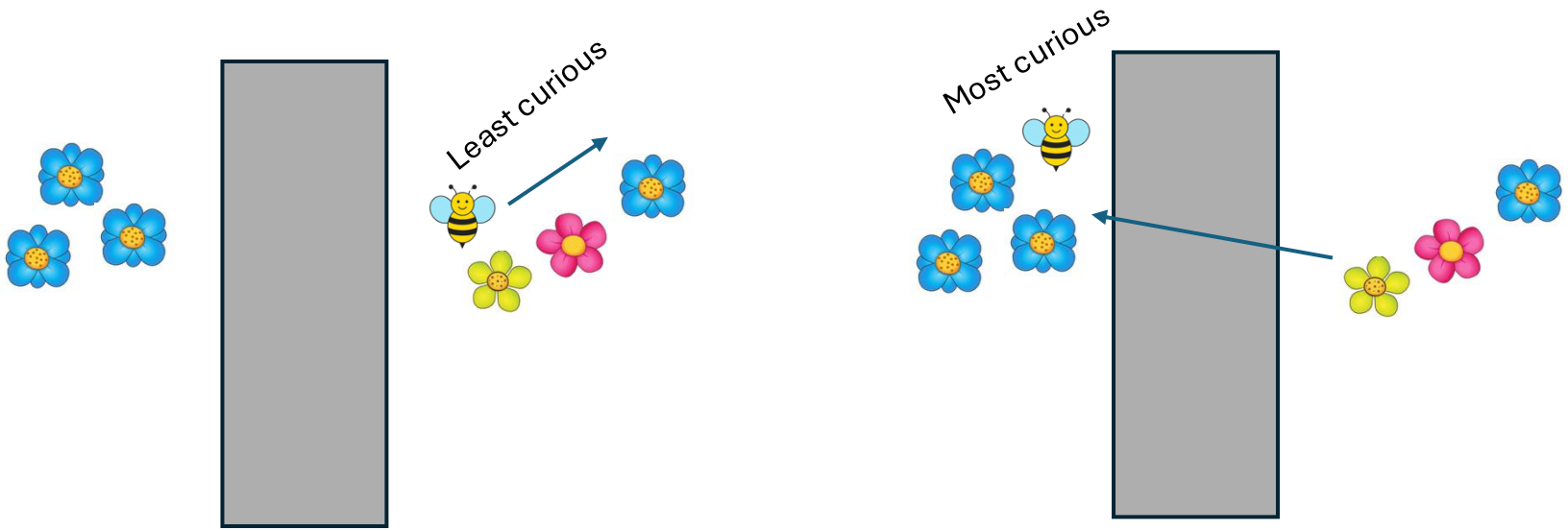
Earl, D.J. & Deem, M.W. (2005). "Parallel Tempering: Theory, Applications, and New Perspectives." *Physical Chemistry Chemical Physics*, *7*, 3910–3916.

Population samplers—parallel tempering

In a parallel termping algorithm imagine we have a family of bees with different curiousity levels



The least curious bees only follow pollen gradient like in MH
The most curious don't care about pollen at all and will just go wherever they like.



What then happens is the most curious bee tells the least curious about it's new find and so all the family can migrate over

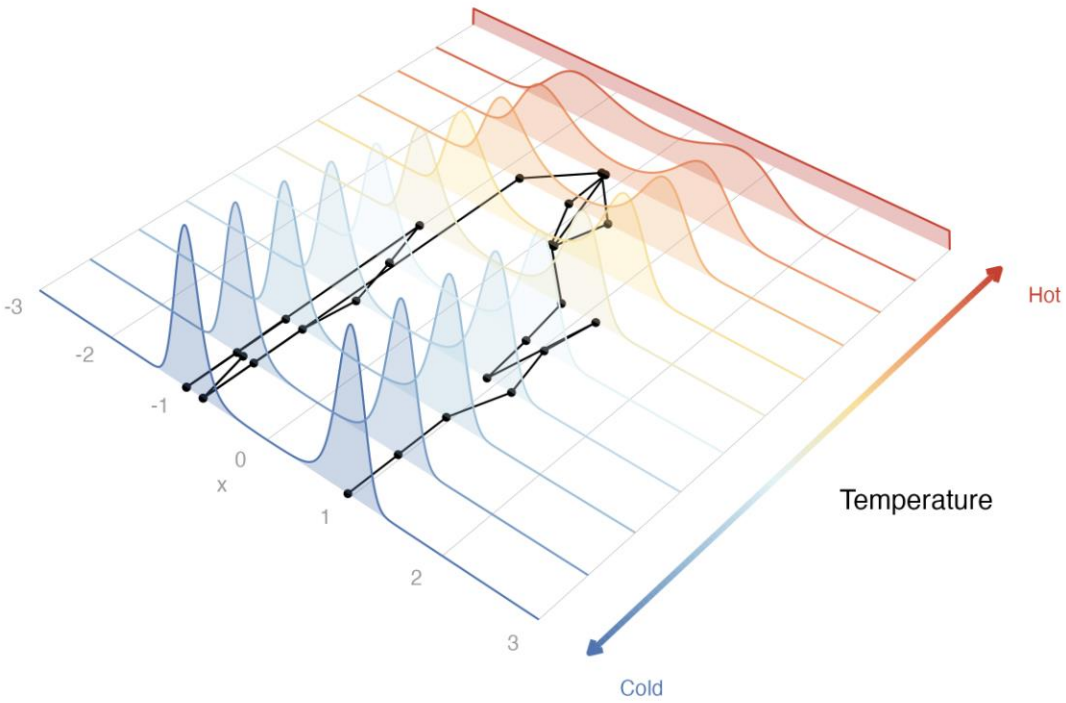
Very useful in highly multimodal likelihoods

Population samplers—parallel tempering

In parallel tempering each member of the bee family is refered to as a ladder.
The posterior for each element, *j,* of the ladder is defined by

$$\pi_j = \pi(X|y)^{1/T_j}, \text{ where } T_0 = 1 < T_1 < \cdots < T_n$$

cold hot



Each ladder of the chain has it's own markov chain defined as before (for when at state *i)*

$$\alpha = \min\left(1, \frac{p(X^{j,\prime} \mid y)}{p(X_i^j \mid y)}\right)$$

To communicate between temperature ladders there's also an additional acceptance ratio for swapping chains between ladder given by

$$\alpha_t = \min\left(1, \frac{\pi_j(X_i^j)\pi_{j+1}(X_i^j)}{\pi_j(X_i^{j+1})\pi_{j+1}(X_i^{j+1})}\right)$$

Usually just log the **coldest** chain when drawing the posterior.

Great for handling multimodal spaces. Linear increase in run length (N = 10 then 10x longer)

Population samplers—other types

1. Differential Evolution MCMC (DE-MC)

Runs N chains, each with state x_i. Proposals are generated using the difference between two *other* randomly chosen chains so no need to tune a proposal covariance manually.

$$x_i^* = x_i + \gamma\left(x_{r_1} - x_{r_2}\right) + \epsilon$$

where $r_1,\ r_2$ are two other chains sampled without replacement, γ is a scale factor.

$$\alpha = \min\left(1, \frac{\pi(x_i^*)}{\pi(x_i)}\right)$$

Why it's useful: the proposal automatically adapts to the shape/scale/orientation of the target because it's built from the current spread of the population — great for correlated, awkwardly-shaped posteriors. **DE-MCz** and **DREAM** are popular extensions.

1. ter Braak, C.J.F. & Vrugt, J.A. (2008). "Differential Evolution Markov Chain with Snooker Updater and Fewer Chains." *Statistics and Computing*, 18(4), 435–446

2. Multiple-Try Metropolis (MTM)

At each step, propose k candidate points from the current state, evaluate all of them, and select one with probability proportional to a weight (e.g. π-based):

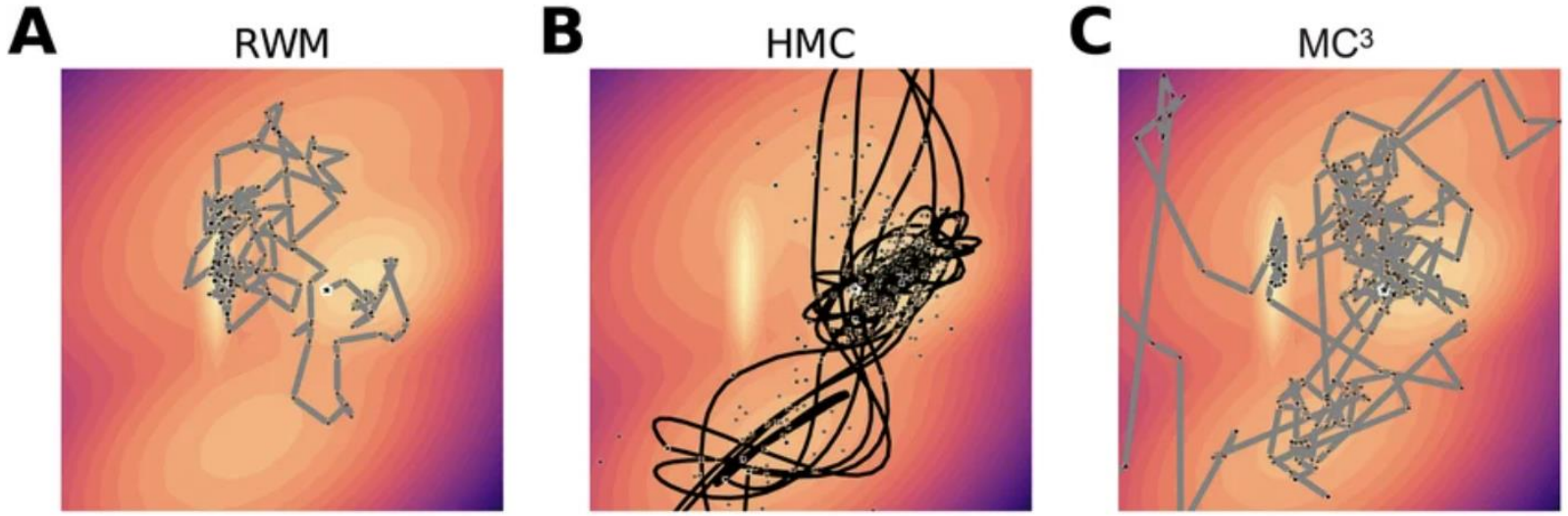
$$w(x_j) = \pi(x_j)\, q(x, x_j)$$

Then correct with a generalised MH acceptance ratio that references a matching set of "reference" points from the reverse move. Increases the chance of a good move at the cost of more density evaluations per step — parallelizes well since the k candidates can be evaluated simultaneously.

2. Liu, J.S., Liang, F. & Wong, W.H. (2000). "The Multiple-Try Method and Local Optimization in Metropolis Sampling." *Journal of the American Statistical Association*, 95(449), 121–134.

https://doi.org/10.1371/journal.pcbi.1010312.g002

Summary of main samplers:



Method	Pros	Cons
Metropolis–Hastings (basic RW)	Simple to use, quick	Limited practicle use one you get >2 dimensions
Adaptive Metropolis–Hastings	Simple to use, quick Scale well in high dimensions	Can get stuck in highly multimodal spaces
Hamiltonian Monte Carlo (incl. NUTS)	Extremely flexible + fast for most MCMC sampling systems Lots of community support and supportive packages Language agnotic versions Scale very well in higher dimensions	Harder to use (*stan* dominants and has it's own PPL) Diagnostics are more involved
Parallel Tempering	Deals with multimodal spaces well Most inference happens under the hood so quite easy to use	Harder to find good implementations Can be very slow

CHARITÉ
UNIVERSITÄTSMEDIZIN BERLIN

3. Implementation and diagnostics

We have our samples from the posterior, but how do we know they are right?

Implementation methods

Method	R packages	Python packages
Metropolis–Hastings (basic RW)	`mcmc::metrop`, `MCMCpack::MCMCmetrop1R`, `LaplacesDemon` (Algorithm="MWG"/"RWM"), `nimble` (default RW sampler)	Custom/`PyMC` (`pm.Metropolis`), `TensorFlow Probability` (`tfp.mcmc.RandomWalkMetropolis`), `emcee` (via `moves.MHMove`)
Adaptive Metropolis–Hastings	`adaptMCMC` (Haario et al. AM), `FME::modMCMC`, `BayesianTools` (sampler="Metropolis", adapt=TRUE), `nimble` (adaptive RW is default), `LaplacesDemon` (Algorithm="AM"/"RAM"), `monty`	`PyMC` (`pm.Metropolis` auto-tunes scale during tuning phase), `NumPyro`? (mostly HMC-focused, but has `BarkerMH`), custom AM via `numpy`/`scipy`, `PyMC3` DEMetropolisZ
Hamiltonian Monte Carlo (incl. NUTS)	`rstan`/`brms`/`rstanarm` (Stan backend, NUTS), `nimbleHMC`, `hmclearn`, `monty`	`Stan` (`PyStan`, `CmdStanPy`), `PyMC` (`pm.HamiltonianMC`, default NUTS), `NumPyro` (`NUTS`, `HMC`), `TensorFlow Probability` (`tfp.mcmc.HamiltonianMonteCarlo`, `NoUTurnSampler`), `Blackjax`
Parallel Tempering	`mcmc::temper` (Geyer's $(MC)^3$), `ptmc` (github, general-purpose PT-MCMC), `nimbleAPT` (adaptive PT for nimble), `BayesianTools` (Metropolis + tempering function), `monty`, `ptmc`	`ptemcee` (fork of emcee's PT sampler), `PTMCMCSampler` (NANOGrav, used widely in GW/pulsar astro), `emcee` legacy `PTSampler` (deprecated), `TFP` (`tfp.mcmc.ReplicaExchangeMC`)

Summarised many of the samplers in R and their packages here: https://dchodge.github.io/rmcmcatlas/

Diagnostics—key notation

If we have a sample from a MCMC sampler X_i, then we known that $X_i \rightarrow \pi(\mathrm{X} \mid \mathrm{data})$ as $i \rightarrow \infty$. But we cannot wait forever.

So when is i big enough to stop? i.e. how long do we need to run i for the samples to have converged.

NO WAY OF KNOWING A PRIORI FOR SURE IN COMPLEX SYSTEMS.

So we turn to run four chains separately (or 4 ladders in the parallel termpering case) and see if they end up sampling from the same region.

Then we calcualte metrics to determine if they the samples are in the same place and also assess how well the chains are mixing.

Diagnostics—key notation

If we run a chain for *M* steps, X_i.
Chain length: *M*

Burn-in: ususally we don't include all M steps as the start of the chain might be in a very unusal places (tiny likelihood) which we woud never sample from naturally. To prevent this editing the chain we throw away the first M/2 samples.

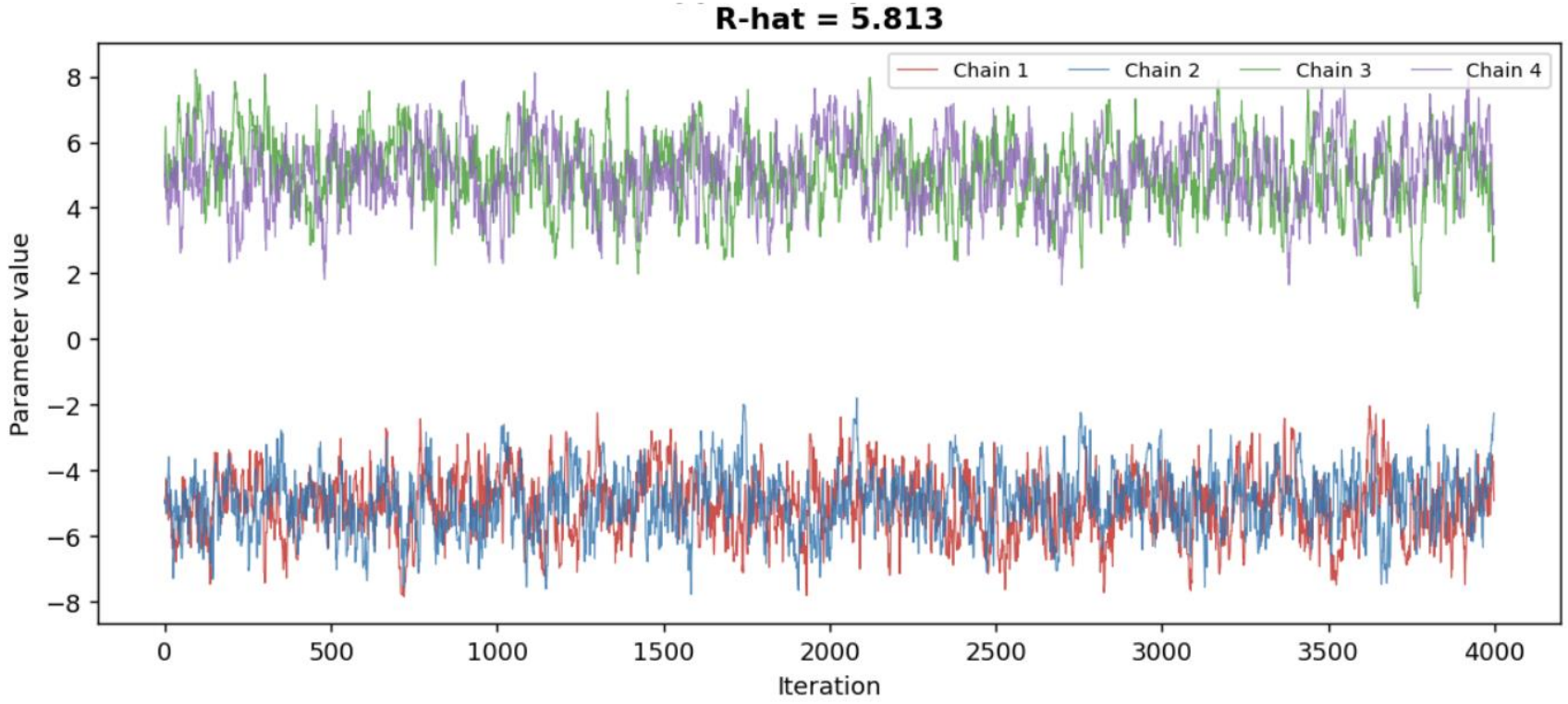
Thinning: After removing burn in you can throw away every p point. Why do this?

- Help reduce the number of samples if too large to deal with computationally (usuallly ai for 1,000 overall)
- Helps reduce autocorrelation between samples. If chain is struggling to sample then lots of values will be simimilar which are close to each other adding marginal information.

Acceptance rate: number of accepted chains/length of chain. Want to be around 0.234

Rhat, Potential Scale Reduction Factor (PSRF), Gelman-Ruban statistic: compare the variance *within* each chain (W) to the variance you'd estimate by pooling *across* chains (which includes between-chain variance B). If converged should be < 1.01.

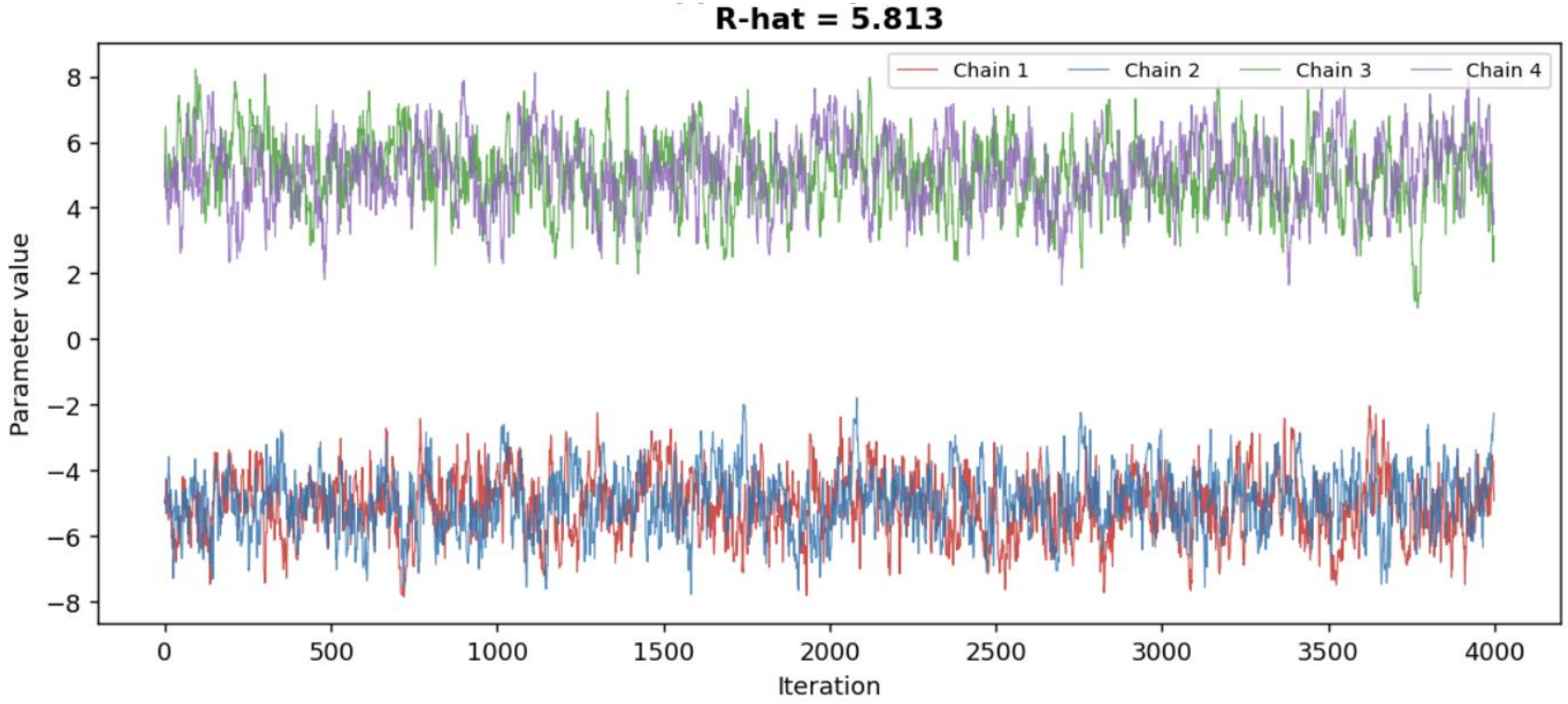
Diagnostics—Example 1



Chain	Accept. rate	Mean log-lik (2nd half)	Chain mean
1	0.82	-0.50	-5.14
2	0.82	-0.44	-4.80
3	0.82	-0.56	4.90
4	0.82	-0.58	5.12

Target: 50/50 mixture N(-5,1) & N(5,1). Local proposal (sd=0.6) cannot cross the valley between modes.

Diagnostics—Example 1



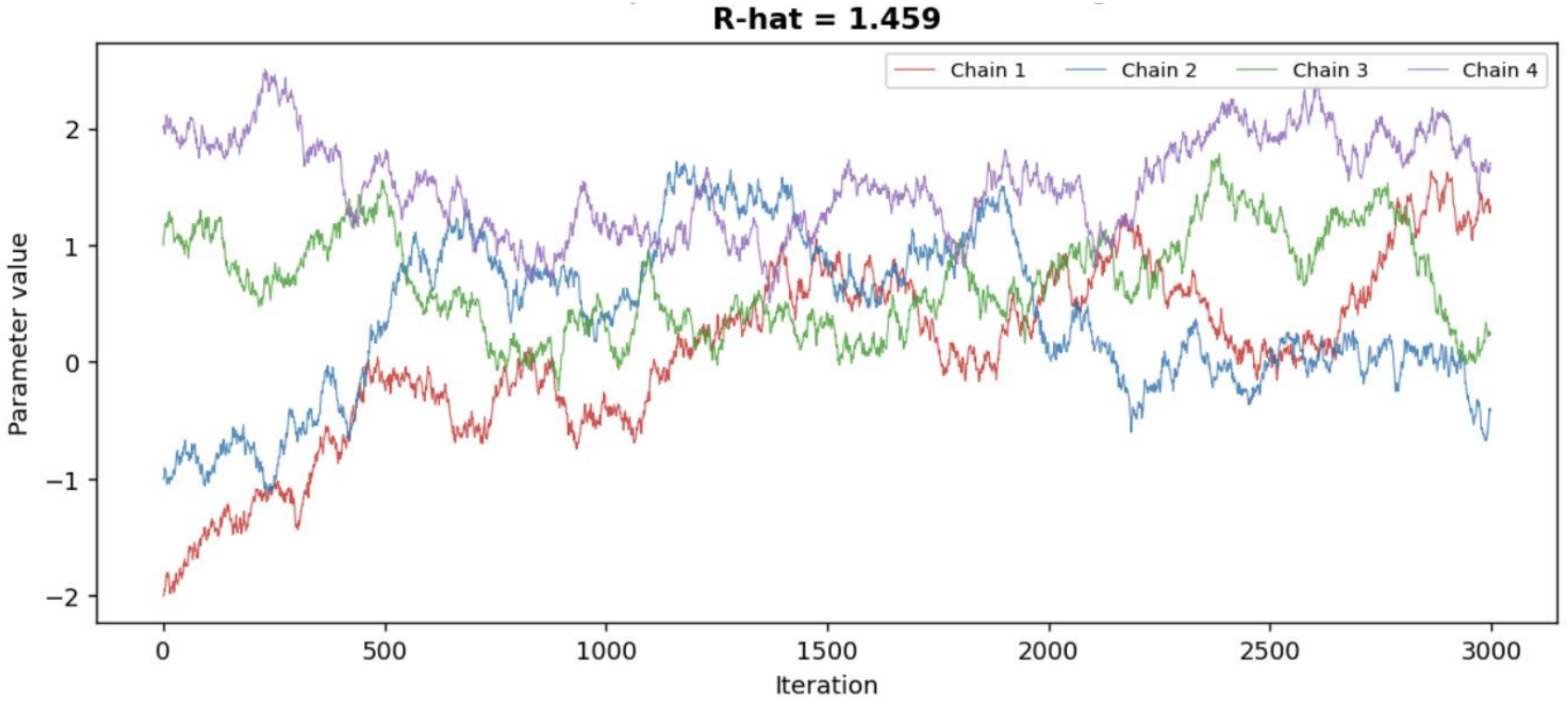
Chain	Accept. rate	Mean log-lik (2nd half)	Chain mean
1	0.82	-0.50	-5.14
2	0.82	-0.44	-4.80
3	0.82	-0.56	4.90
4	0.82	-0.58	5.12

Target: 50/50 mixture N(-5,1) & N(5,1). Local proposal (sd=0.6) cannot cross the valley between modes.

Multimodal

- Looks like mode ~5 has higher likelihood
- Use a parallel tempering or something that helps get over multimodal spaces

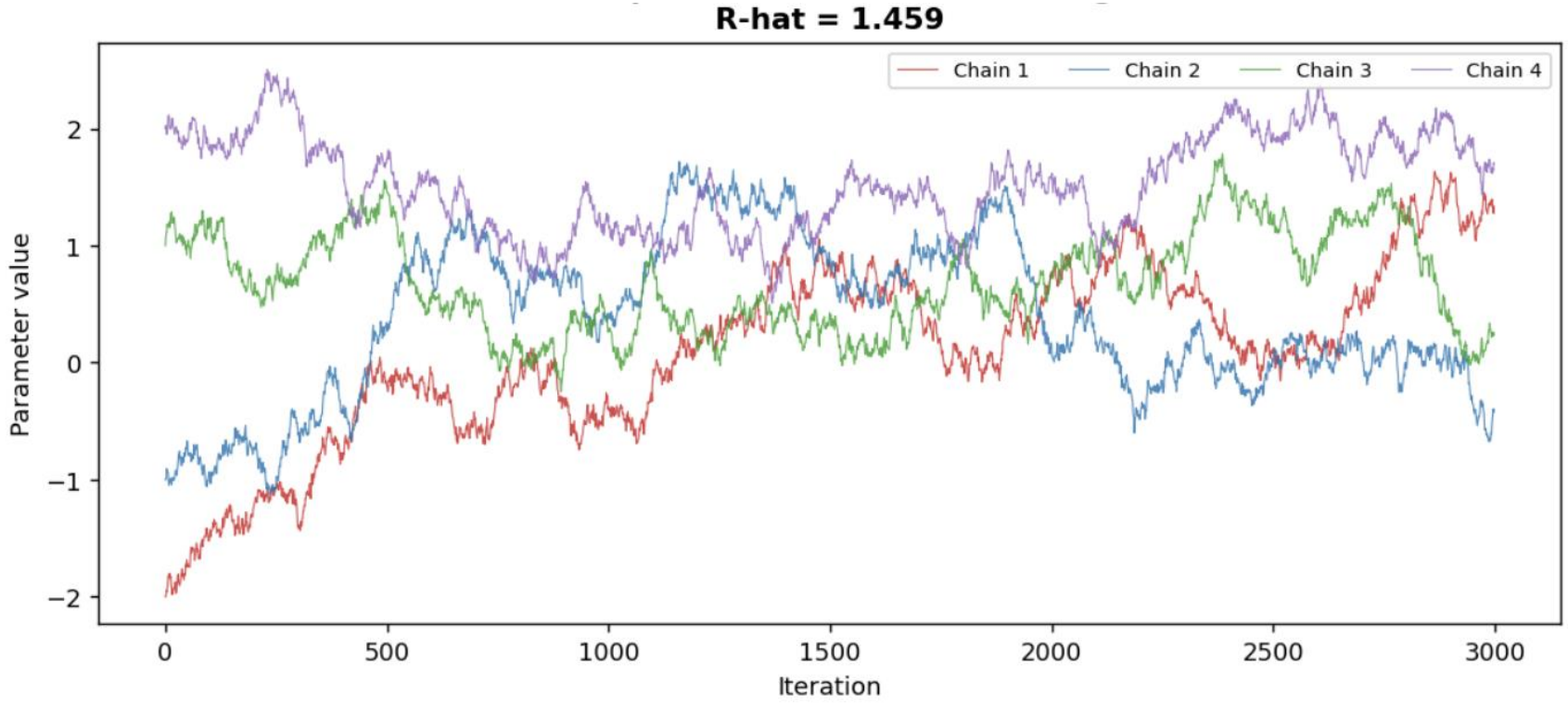
Diagnostics—Example 2



Chain	Accept. rate	Mean log-lik (2nd half)	Chain mean
1	0.99	-0.27	0.59
2	0.99	-0.15	0.28
3	0.99	-0.41	0.80
4	0.97	-1.44	1.66

Target: N(0,1). Proposal sd=0.04 gives ~95% acceptance but chains crawl -> high autocorrelation, insufficient exploration in the run length shown.

Diagnostics—Example 2

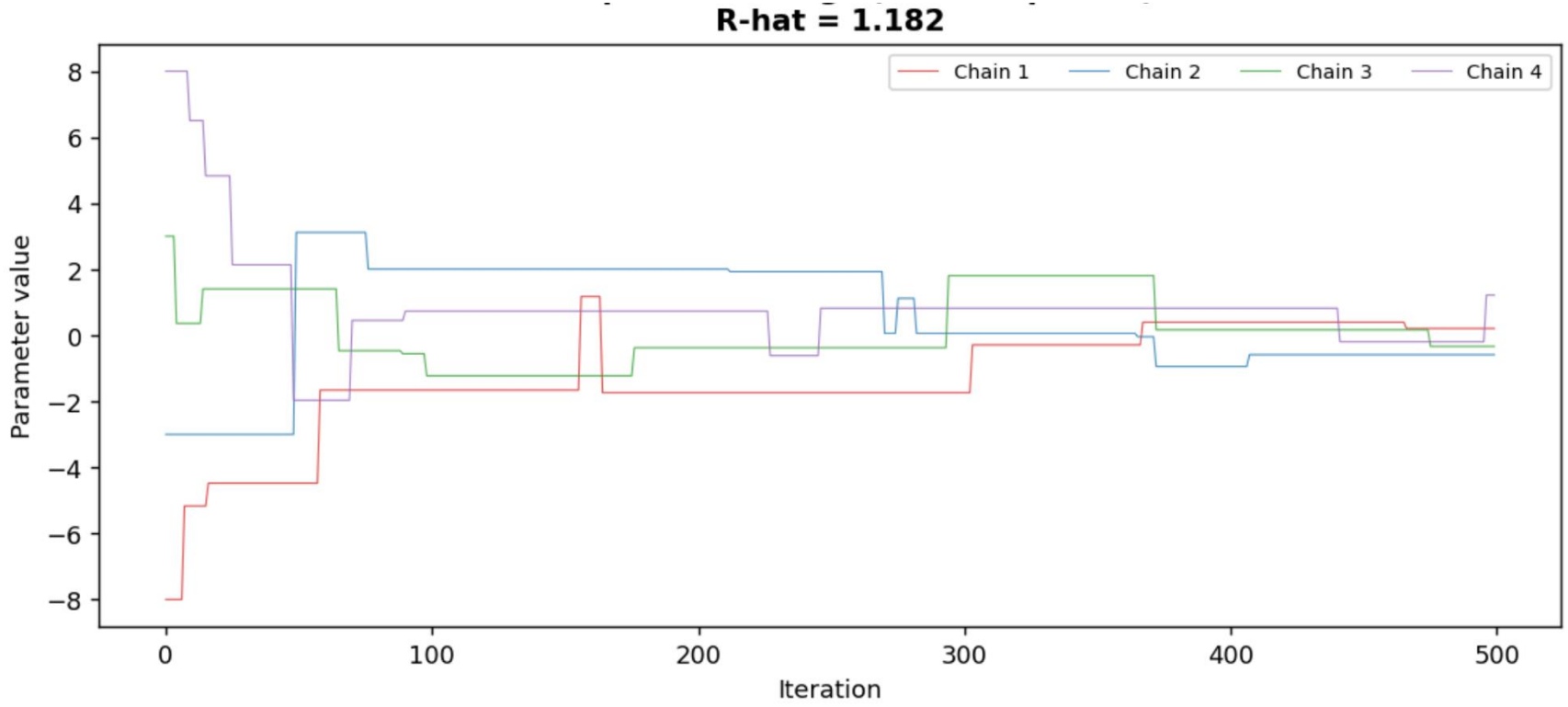


Chain	Accept. rate	Mean log-lik (2nd half)	Chain mean
1	0.99	-0.27	0.59
2	0.99	-0.15	0.28
3	0.99	-0.41	0.80
4	0.97	-1.44	1.66

Target: N(0,1). Proposal sd=0.04 gives ~95% acceptance but chains crawl -> high autocorrelation, insufficient exploration in the run length shown.

Step-size too small
- Acceptance rate too high
- Use adaptive MCMC or increase step size

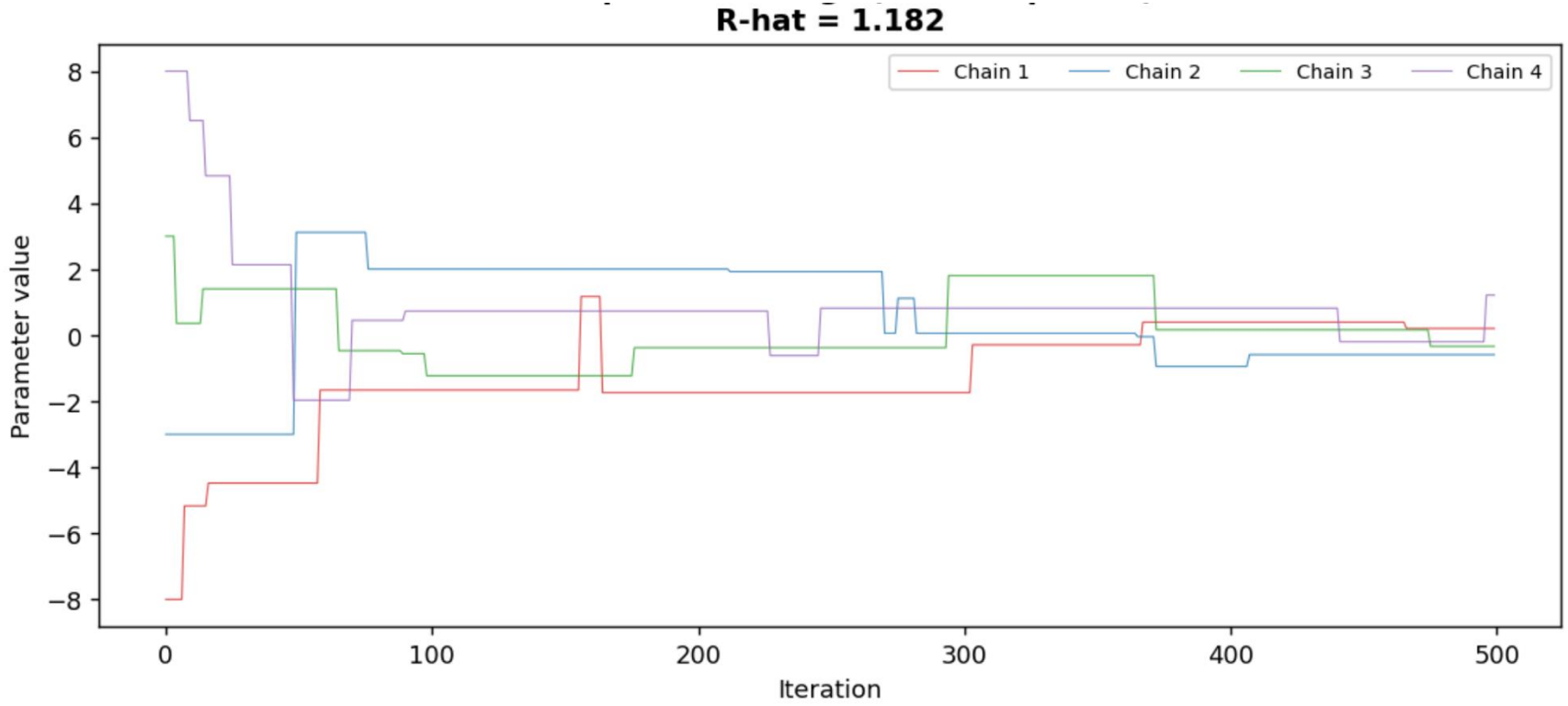
Diagnostics—Example 3



Chain	Accept. rate	Mean log-lik (2nd half)	Chain mean
1	0.02	-0.36	-0.26
2	0.02	-0.29	-0.14
3	0.02	-0.53	0.53
4	0.02	-0.27	0.60

Target: N(0,1). Proposal sd=60 -> almost all jumps rejected, chains freeze near their start for long stretches at very different values.

Diagnostics—Example 3



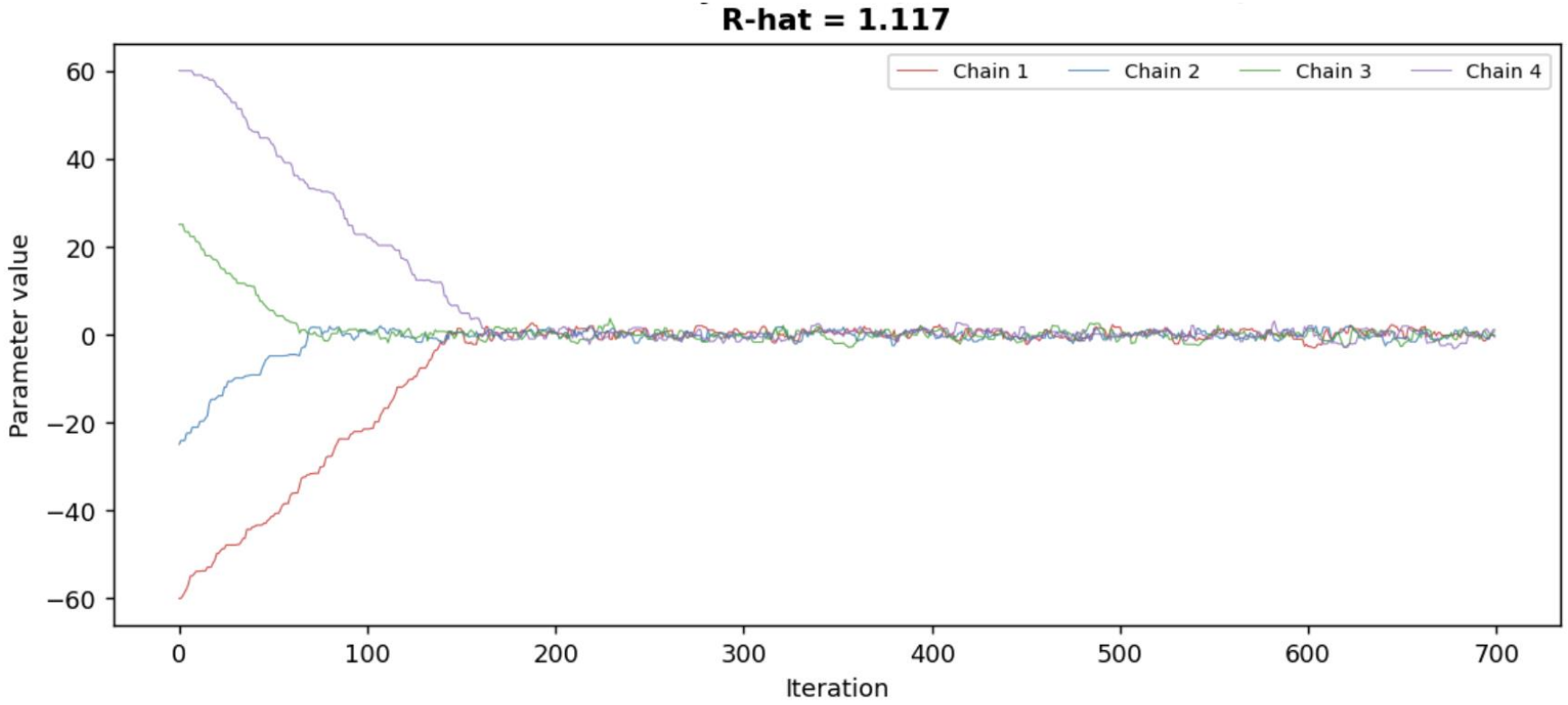
Chain	Accept. rate	Mean log-lik (2nd half)	Chain mean
1	0.02	-0.36	-0.26
2	0.02	-0.29	-0.14
3	0.02	-0.53	0.53
4	0.02	-0.27	0.60

Target: N(0,1). Proposal sd=60 -> almost all jumps rejected, chains freeze near their start for long stretches at very different values.

Step-size too large

- Acceptance rate too low
- Rhat is not bad
- Use adaptive MCMC or decrease step size

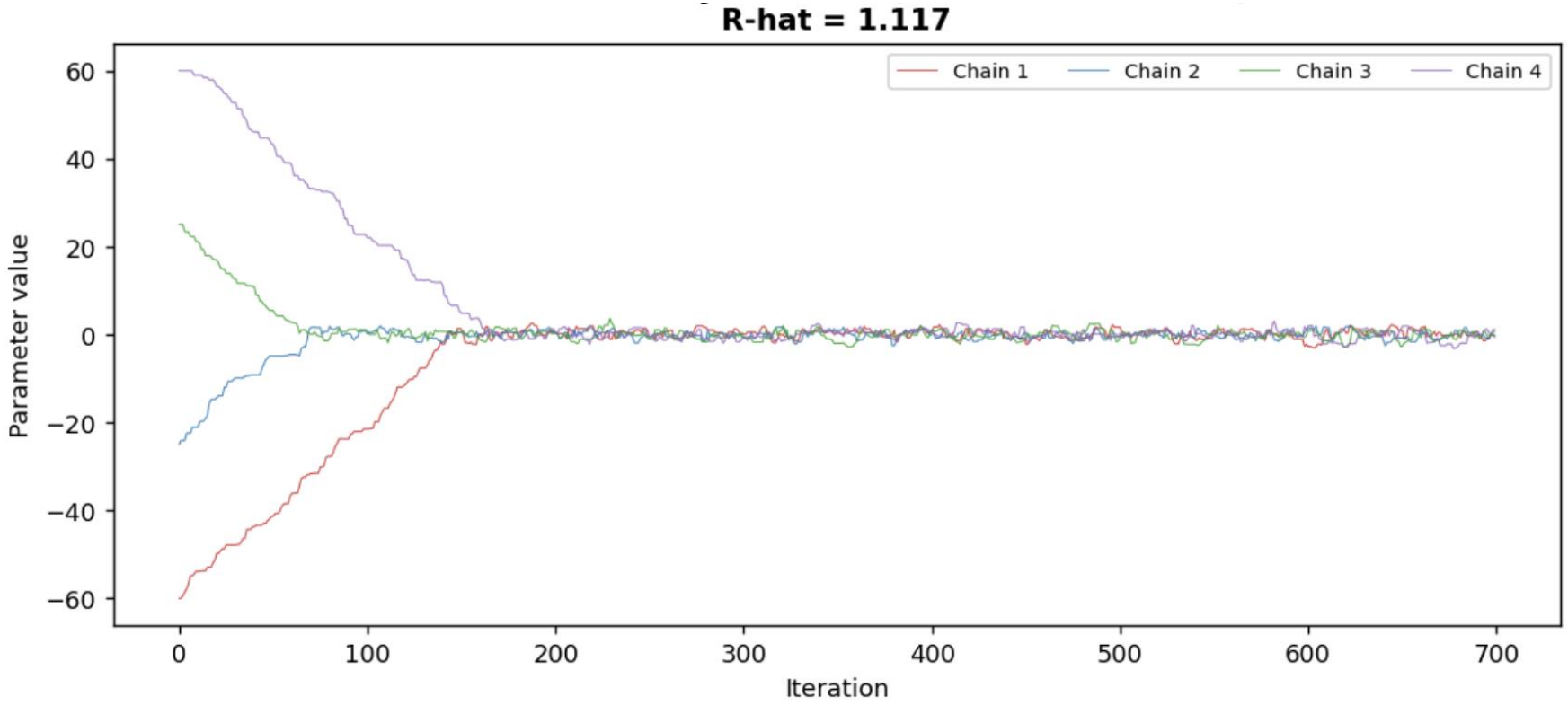
Diagnostics—Example 4



Chain	Accept. rate	Mean log-lik (2nd half)	Chain mean
1	0.65	-0.54	0.10
2	0.67	-0.40	-0.17
3	0.71	-0.61	-0.19
4	0.66	-0.66	0.03

Target: N(0,1). Chains start far from the typical set; kernel mixes fine (sd=1) but the run is stopped before the transient has fully decayed -> means still differ.

Diagnostics—Example 4

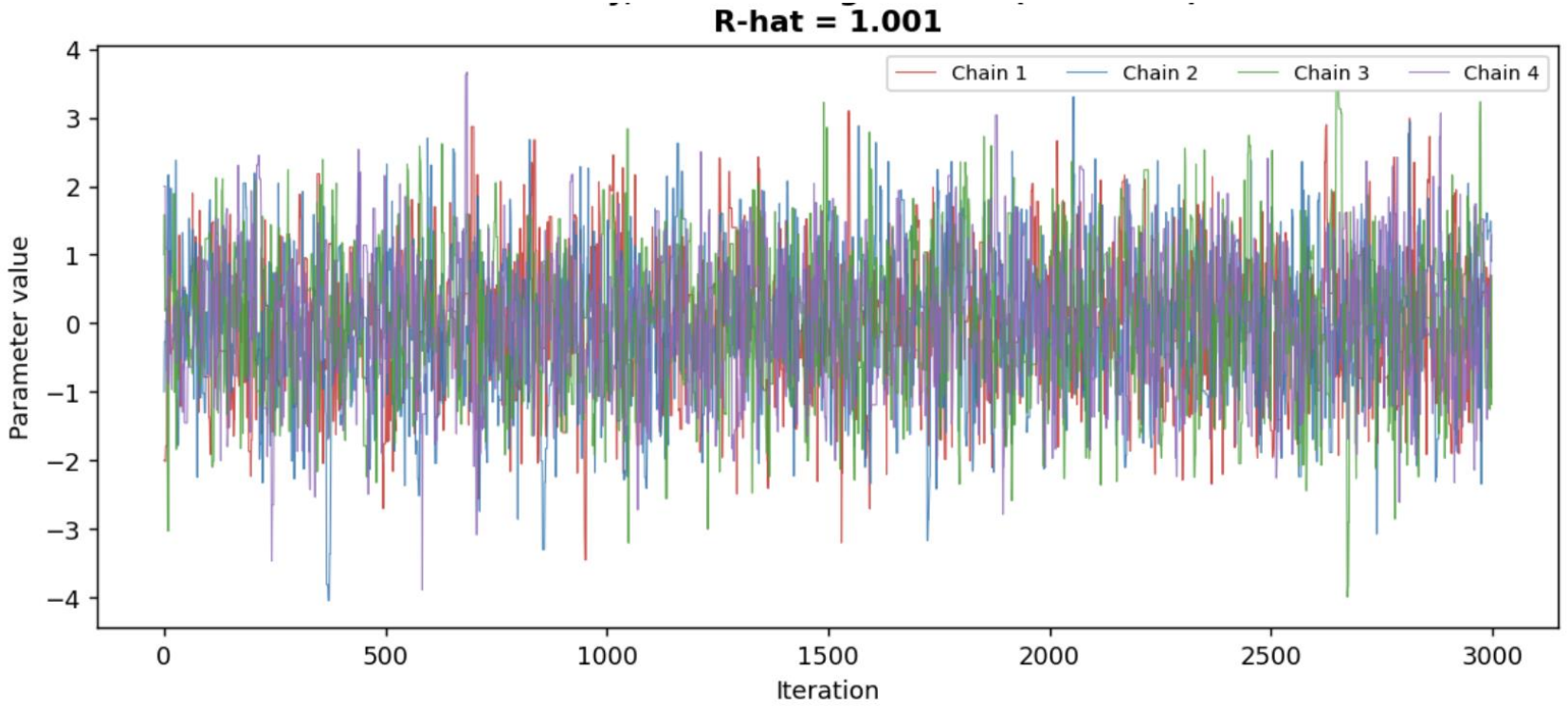


Chain	Accept. rate	Mean log-lik (2nd half)	Chain mean
1	0.65	-0.54	0.10
2	0.67	-0.40	-0.17
3	0.71	-0.61	-0.19
4	0.66	-0.66	0.03

Target: N(0,1). Chains start far from the typical set; kernel mixes fine (sd=1) but the run is stopped before the transient has fully decayed -> means still differ.

Longer burn in needed
- Seems to converge post 200 samples

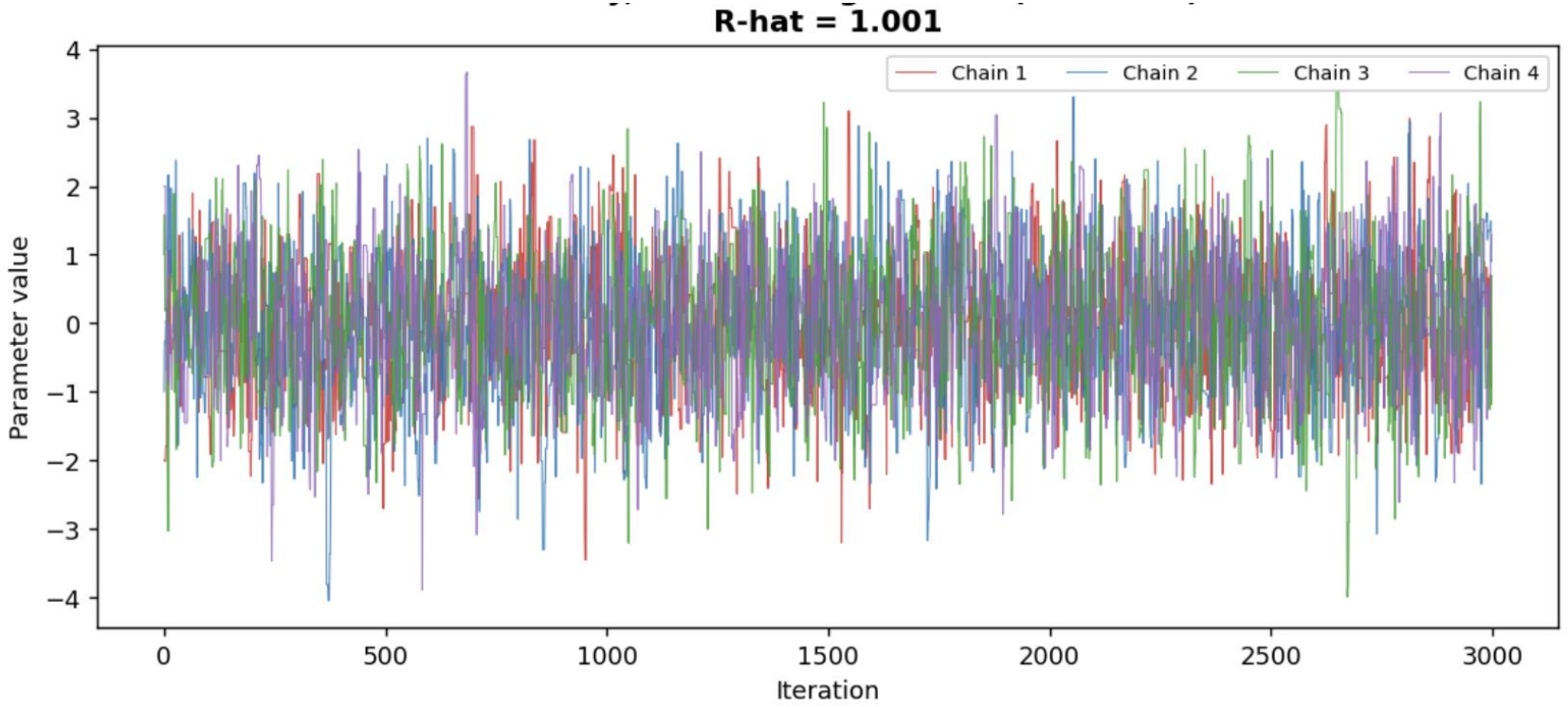
Diagnostics—Example 5



Chain	Accept. rate	Mean log-lik (2nd half)	Chain mean
1	0.43	-0.46	-0.06
2	0.46	-0.54	0.11
3	0.44	-0.59	0.09
4	0.43	-0.48	0.05

Target: N(0,1). Proposal sd=2.4 tuned for ~30-40% acceptance; dispersed-but-reasonable starts; chains overlap and explore the full support -> Rhat approx 1.

Diagnostics—Example 5

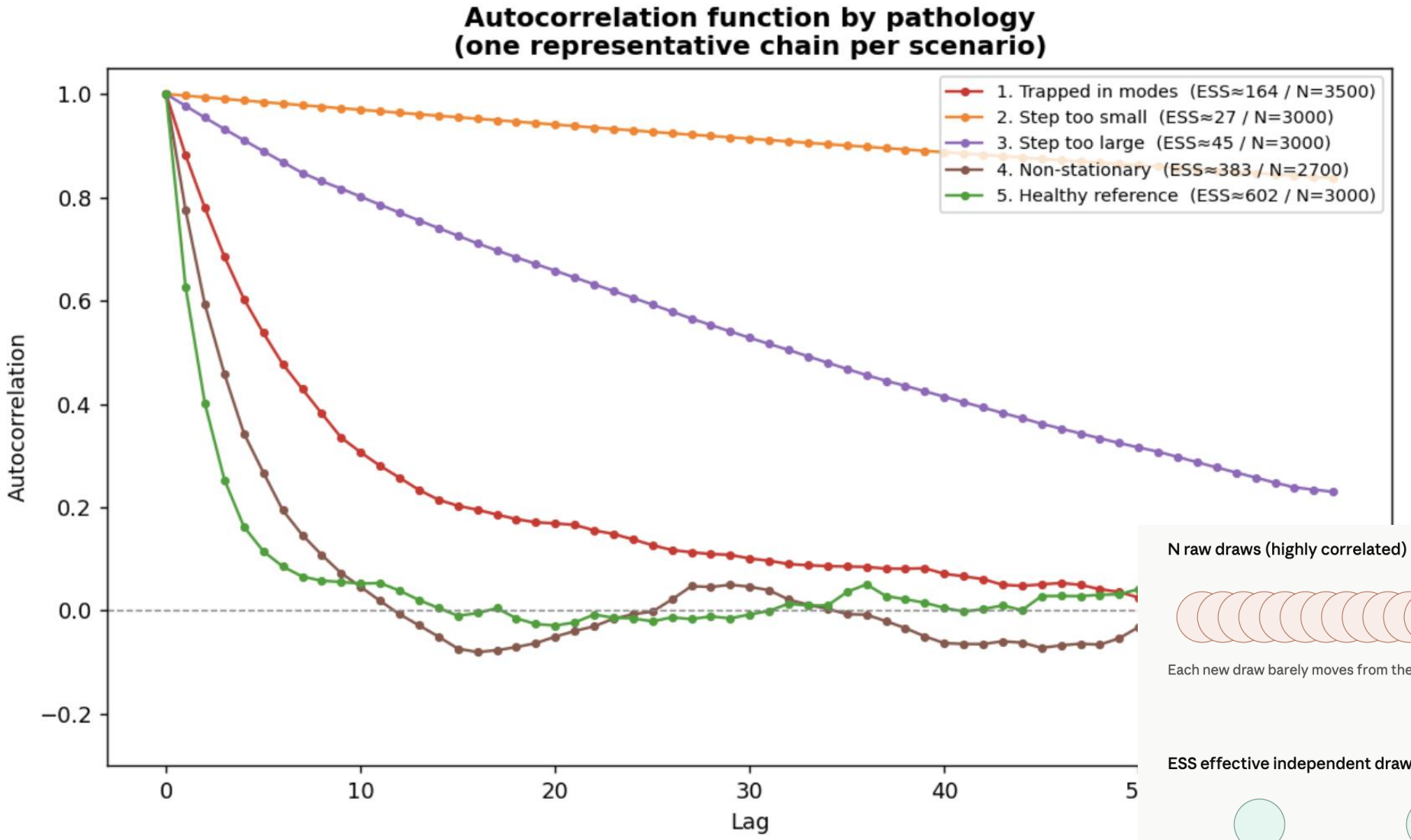


Chain	Accept. rate	Mean log-lik (2nd half)	Chain mean
1	0.43	-0.46	-0.06
2	0.46	-0.54	0.11
3	0.44	-0.59	0.09
4	0.43	-0.48	0.05

Target: N(0,1). Proposal sd=2.4 tuned for ~30-40% acceptance; dispersed-but-reasonable starts; chains overlap and explore the full support -> Rhat approx 1.

Looks good!

Diagnostics—Autocorrelation, chain efficiency

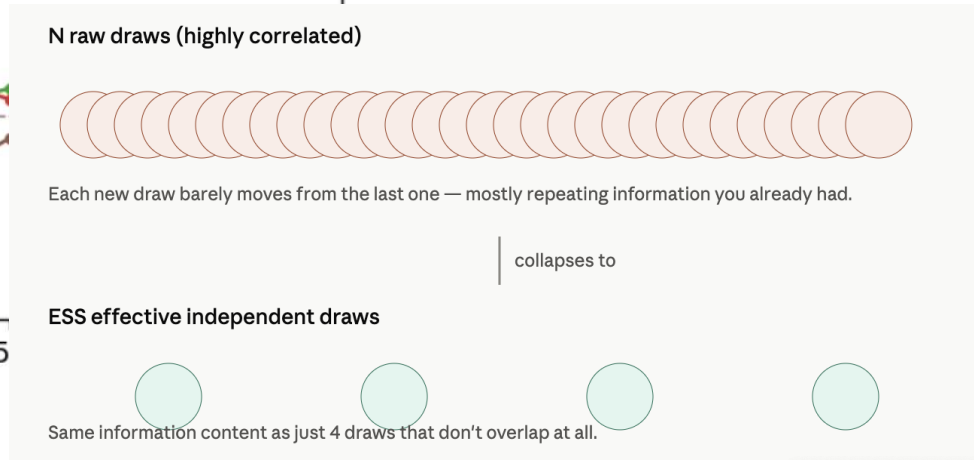


Correlatoin between the chains k steps later

Want ESS close to sample number as possible

Having ESS ~300 for 1000 samples considered very good

Very low ESS suggests very little information in the sample



Diagnostics—Summary

Usually risky to rely on one metrics to determine convergence:

- But if you had to **Rhat is the most reliable, if <1.01 for all parameters** then fair justification for convergence.

- But should always check the traceplots for every parameter and the ESS as can give you information about how good the sampler is working
  - If things are looking bad for some parameters needs to rethink sampler of model structure
  - Even if things are looking good can potential reduce run and make modelling quicker

Also check the acceptance rate—may suggest different sampler is better andhelp diagnose issues

Checking likelihood if you have multimodel systems can be useful

CHARITÉ
UNIVERSITÄTSMEDIZIN BERLIN

4. Exotic samplers

Light overview of more specific implementations of MCMC

Gibbs sampling

Special case of Metropolis-Hasting MCMC

WHAT THIS IS

- At each step it samples one parameter value at a time (even in high dimensions spaces) keeping all the others the same
- But it uses the full conditional distribution to sample instead of accept/reject step
- Thus every proposed draw is accepted as it's from the conditional distribution
- **Need** full conditionals in closed form

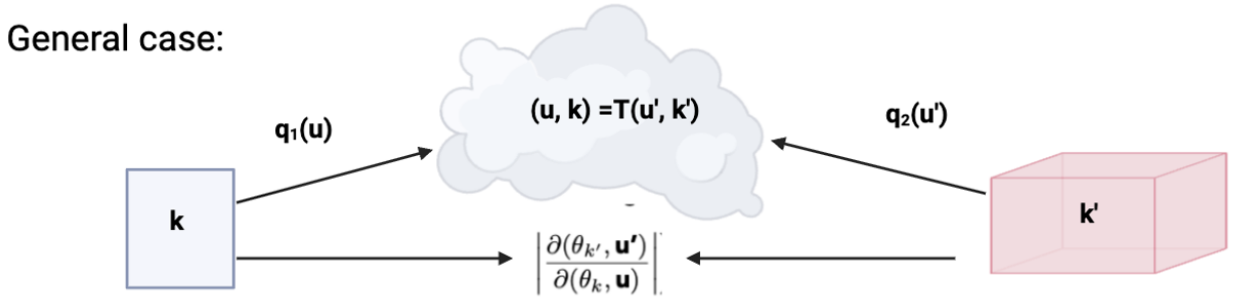
RARELY USEFUL IN ID MODELS

- Infectious disease models involve latent structure—unobserved infection times tranemission, etc—hard to find
- Likelihood is build from a dynamical process, not exchangable, sop onditional rarally reduce to standard distributions
- Attractive to use in simple cases, but ususally inapplicable without heavy approximation or data augmentation

Always crops up in teaching material as incredible efficient but isn't super useful for complex models
There is a thing called **Metropolis-within-Gibbs** where you are Gibbs sampling (one parameter at a time) but using MCMC instead of the full conditional distribution

Reversible Jump MCMC

Sampling across different dimensions



The problem it solves

Standard MCMC assumes a fixed number of parameters. Many ID model shave an unknown dimensions.

- Cases in a transmission tree
- Changepoints in transmission rates

How it works

Process 'jumps' between parameters spaces or different dimensions, using auxiliary variables and Jacobian terms so detailed balance holds across dimensions

Why it's useful fo ID

Can infer model structure within the sampling process (instead of having to run many models and compare) and jointly inferred with parameters.
Useful when number of latent events uncertaint

Stengths

- Handles model uncertainty and variable-dimensions paramter spaces naturally
- Avoid haing to fix model compexity in advance

Limitations

- Designing efficient trans-dimensional jump proposals is technically complex and problem-specific (need maths)

Incredibly powerful and lots of theory on how to implement, remain underused

Approximate Methods

When you can't calculate the exact posterior

Motivation: ID models might have intractable likelihood; e.g.

- ABM where things are rule based often cannot calculate the likelihood
- Stochastic systems in general often have no closed form of the likelihood
- Full likelihood calculation is very time intensive

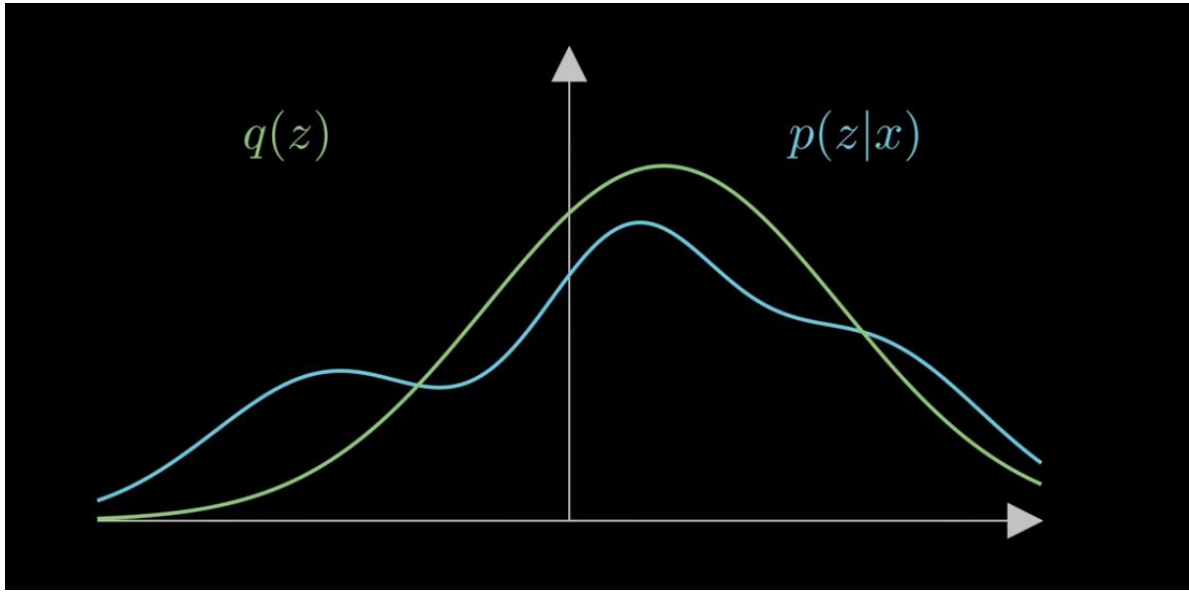
In these cases you can try sample from an approximate of the likelihood instead of the exact likleihood form

Variational inference

Approximate Bayesian Computation

Variational Inference

Convert to an optimisation problem (bit complicated)



How it works

Say you have a really difficult likelihood space you can't sample from easily, p(y | X).
Pick a simple distriobtuoin q(X), could be Normal, Exp, Gamma. Which best approxiamtes it?

- Use KL divergence to find the best fitting approximate distirbution (via ELBO)

Stengths

- Super fast and scales really well
- Deterministic calculation so no sample and need to assess chain mixing

Limitations

- Ultimately it is wrong (doesn't mean it's not useful)
- Uncertainty is usually poorly captured
- Accuracy dependents heavily on chosen approximating families

- Good for fast initial exploration of space and also interesting hybrid models with MCMC samplers

Approximate Bayesian Computation (ABC)

Likelihood-free inference by simulation

Propose

Draw parameters from a prior

→

Simulate

Generate synthetic data from the model under those parameters

→

Compare

Reduce simulated + observed data to summary statistics and measure their distance

→

Accept/Reject

Keep the parameters is the distance is below a tolerance ε

Stengths

- No likelihood—just a forward simulator
- Flesible: aplpies to almost any generative model

Limitations

- Accuracy to truth posterior is sensitive to choice of summary statistics and tolerance
- Computationally intensive — requires very many simulations

- In likelihood-free systems: no statistical mapping from ***parameters to probability (does this matter?)***

To summarise

Hopefully you learn today:

1. What MCMC samplers are and why we need them
2. Have a decent understanding of what each of these samplers do
    - Metropolis-Hasting
    - Adaptive Metropolis-Hasting
    - Hamiltonian Monte Carlo
    - Parallel tempering
3. Be able to diagnose convergence of 4 chains
4. Have some rough idea of exotic samplers and what they do

Hopefully inspires you to get into MCMC sampling and make it less scary !!

(Didn't have time to make slides on SMC, including particle filters and ABC-SMC sorry, can add to slides if useful)